Technology
Arts Sciences
TH Köln

Entwicklungsprojekt interaktiver System

Modellierungsbegründungen

Christopher Huntscha, Sören Lehmann

16.12.2018

Inhaltsverzeichnis

1. Vorgehensmodell **4**
1.1 Nachtrag: Abwägung 4
1.2 Usability Engineering Lifecycle 4
1.2 ISO 9241 - Teil 110 6
1.3 Fazit 6

2. Modellierung anhand der Top-Down Methode **7**

3. Iteration der Zielhierachie **8**
3.1 Strategisches Ziel 8
Operatives Ziel 8
Taktisches Ziel 8
Operatives Ziel 8
3.2 Strategisches Ziel 8
Taktisches Ziel 8
Operatives Ziel 8
3.3 Strategisches Ziel 9
Taktisches Ziel 9
Operatives Ziel 9
3.4 Fazit 9

4. User Profiles **10**
4.1 Informatikstudent 10
4.2 Schüler mittlerer Schulreife 11
4.3 Professor der Informatik 12
4.4 Fazit 13

5. Contextual Task Analysis **14**
5.1 Brief Use Cases 14
5.2 Task Szenarien 16
5.2.1 Task: Anlegen von Karteikartenstapeln durch Karteikarten anderer Nutzer 16
5.2.2 Task: Auswahl eines Karteikartenstapels für die aktuelle Lerneinheit 17
5.2.3 Task: Das Hinzufügen eines Karteikartenstapels zu den eigenen Favoriten 17
5.2.4 Liste von weiteren möglichen Task Scenarios 18
5.3 Current User Task Organization Model 19

6. Usability Goals **20**
6.1 Qualitative Usability Goals 20
6.1.1 Ziel 1: Die grafische Gestaltung der Anwendung soll für den Nutzer leicht verständlich sein 20
6.1.2 Ziel 2: Der Nutzer soll eigene Karteikarten schnell anlegen können, um den Lernprozess zu optimieren 20

6.1.3 Ziel 3: Der Nutzer soll eigene Karteikartenstapel einfach und schnell erstellen können 20
6.1.4 Ziel 4: Die Nutzerschnittstelle sollte selbsterklärend sein, damit Spaziergänge sofort mit einer Lerneinheit gestartet werden können 21
6.1.5 Ziel 5: Das System sollte die Nutzer authentifizieren können 21
6.1.6 Ziel 6: Die Spazier-Route sollte schnellstmöglich angezeigt werden 21
6.1.7 Ziel 7: Das Design sollte die Nutzer dazu animieren das System öfter zu nutzen 22
6.1.8 Ziel 8: Die vom Professor eingereichten Karteikartenstapel sollten markiert werden 22
6.2 Quantitative Usability Goals 23
6.3 Fazit 25

7. Platform Capabilities and Constraints 26

8. Gestaltungsprinzipien 28
8.1 “Die Navigation in UWP-Apps basiert auf einem flexiblen Modell aus Navigationsstrukturen, Navigationselementen und Funktionen auf Systemebene.” 28
8.2 Cross Platform 28
8.3 Bold, graphical, intentional 28
8.4 Only show what I need when I need it 29

9. Work Reengineering 30
9.1 Use Cases 30
9.2 Iteration Task Scenarios 35
9.3 Reengineered Task Organization Model 37
9.4 Fazit 37

10. Konzeptuelles Modell 38
10.1 Produkt- oder prozessorientiert 38
10.2 Eindeutige Prozess-Identifizierung 38
10.3 Design Präsentation Regeln 38
10.3.1 Desktopanwendung 39
10.3.2 Mobile Anwendung 39
10.4 Ausarbeitung der Conceptual Models 39
10.5 Fazit 41

11. Screen Design Standards 42
11.1 SDS - Mobile Anwendung 42
11.2 SDS - Desktopanwendung 44

12. Detailed User Interface Design 45
12.1 Mobile Anwendung 45
12.2 Desktopanwendung 49

13. Systemdokumentation 51
13.1 Kommunikationsmodelle 51

13.2 Architekturmodell 53
13.2.1 Iteration der Komponenten 53
13.2.2 Kommunikation 54
13.3 Datenbank 55
Redis 55
MySQL 55
MongoDB 56
Abwägung 56
Middleware 56
13.4 Datenstrukturen 57
Karteikarte 57
Karteikartenstapel 57
Nutzer 59
Abwägung 59
13.5 Anwendungslogik 60
Client: Lernender 60
Server 62
Abwägungen 62
13.6 REST 63
13.7 Richardson Maturity Model 65
13.8 Dienste im Web 65
Google Maps 65
REST 65
Abwägung anderer Dienste 65
13.9 Fehlerbehandlung 66
13.10 Endgeräte 66
14. Reflexion 67
15. Quellenverzeichnis 68
16. Abbildungsverzeichnis 68
17. Tabellenverzeichnis 69

1. Vorgehensmodell

Bereits zu Beginn des Projekts hat sich das Team auf benutzerzentrierte Vorgehensmodelle konzentriert, da das Anwendungssystem an eine spezielle Domäne und dessen Benutzergruppe gerichtet ist. Dementsprechend ist es wichtig, dass Aufgaben und Ziele, Nutzungskontexte, sowie Fertigkeiten und Fähigkeiten des Nutzers bei der Konzeption berücksichtigt werden.
Benutzungszentrierte Vorgehensmodelle erfüllen diese Vorgaben nicht im gleichen Ausmaß und heben eher die Optimierung des Nutzungsprozess hervor.

1.1 Nachtrag: Abwägung

Im Konzept wurden bereits andere benutzerzentrierte Vorgehensmodelle, wie das Discount-Usability Engineering und ISO 9241 - Teil 210 Vorgehensmodell beschrieben und grob abgewägt. Da hiermit keine projektspezifische Begründung geboten wurde, wird diese folglich nachgeholt.

Das Discount-Usability Engineering Vorgehensmodell ist für das Anwendungssystem nicht angemessen, da dieses Vorgehensmodell eine geringe Benutzermodellierung im Prototyp Szenario bietet. Ebenfalls bezieht sich dieses Modell auf die 10 Heuristiken von Nielsen, welche aus dem Jahr 1999 stammen und veraltet sind. Der gewählte Domänenraum ist sehr speziell und zieht es vor mit neueren Technologien zu interagieren, weshalb sich nicht für dieses Vorgehensmodell entschieden wurde.

Für die ISO-Norm wird eine Skalierung vorausgesetzt. Hier befindet sich bereits das Problem, da es dem Projektteam an Kompetenz mit den Umgang von Vorgehensmodellen mangelt, um ohne leitende Vorgaben zu einem projektspezifischen Ergebnis zu kommen. Die Evaluationsschritte können zwar zu einem sehr durchdachten Ergebnis führen, sind aber auch zeitintensiv. Da eine Zeitplanung zum aktuellen Stand des Projekts schwierig ist und eine Evaluation eventuell nicht stattfinden wird, wurde sich auch gegen die ISO-Norm entschieden.

1.2 Usability Engineering Lifecycle

Nachdem die Vor- und Nachteile der gegebenen benutzerzentrierten Vorgehensmodelle im Konzept ausgewertet wurden, wurde sich für “The Usability Engineering Lifecycle” (siehe Abb. 1) nach Deborah Mayhew entschieden. In dem Buch bietet Mayhew eine hochgradige Übersicht von dem Aufbau des Vorgehensmodells. Es werden mehrere Beispiele zu den wichtigsten Schritten und Alternativen gezeigt, Aufwände für diese Schritte eingeschätzt und Anfertigungen von Artefakten detailliert beschrieben.

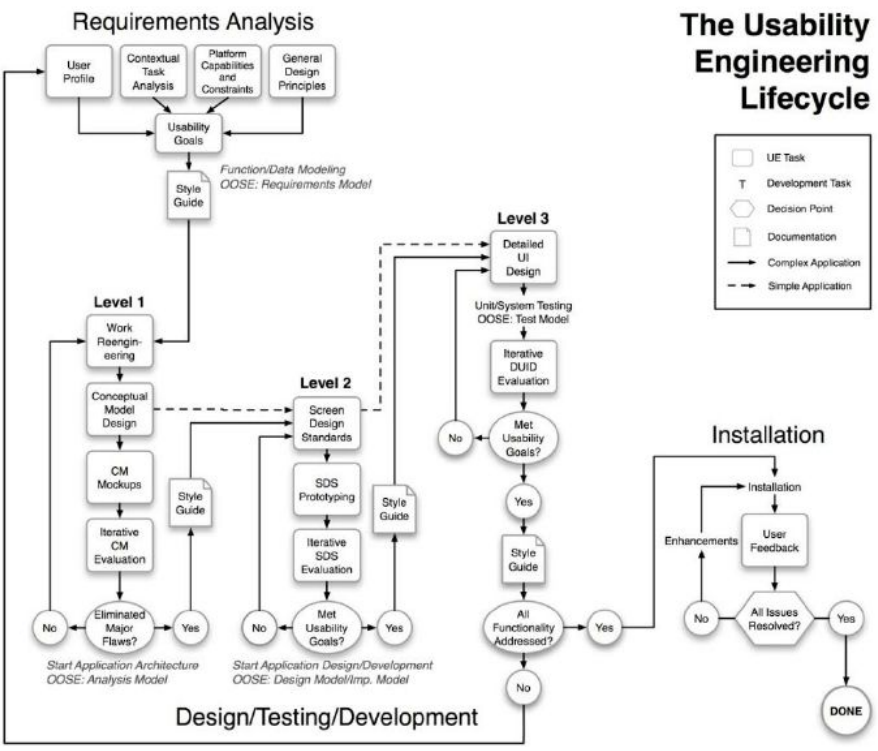


Abb. 1: Usability Engineering Lifecycle nach Deborah Mayhew

Bei der Planung betont die Autorin besonders, dass einfach gestrickte Projekte einer Reduzierung des Lifecycles unterzogen werden können, welche unter anderem das Anwenden von allgemeingültigen Gestaltungsprinzipien, das Zusammenfügen der drei Design Stufen, im Modell mit gestrichelten Linien dargestellt, in einen einfach iterativen Designprozess und das Überspringen des Style-Guides beinhaltet. (Mayhew 1999, S. 5)

Nutzererfahrungen spielen zwar eine wichtige Rolle im weiteren Verlauf des Projekts, können jedoch später im Bereich der Installation realisiert werden. Für die Entwicklung stehen Erfahrungen des Projektteams mit ähnlichen Anwendungen zur Verfügung. Zwar wurde sich auf eine sehr spezielle Domäne geeinigt, jedoch fallen beide Teammitglieder unter die primären Stakeholder (Informatikstudenten), wodurch ein Einbezug der Meinungen relevant ist.

1.2 ISO 9241 - Teil 110

Durch den Lifecycle kann das Projekt sehr flexibel gestaltet werden. Die Autorin nennt in den einzelnen Schritten des Vorgehensmodells mehrere alternative Vorgehensweisen und lässt somit Raum für Erkenntnisse durch abweichende Arbeitsschritte.
Das Projektteam hat sich folglich dazu entschieden Teil 110 der ISO 9241 in die Entwicklung der Dialoggestaltung einzubeziehen. Da es sich hierbei um Grundsätze einer Norm handelt, werden Teilaspekte während der Bearbeitung projektspezifisch runterskaliert. Besonders wichtig für die Entwickler sind die Aufgabenangemessenheit, Selbstbeschreibungsfähigkeit, Erwartungskonformität und Individualisierbarkeit. So soll das Dialogsystem

- den Nutzer bei der Aufgabenbewältigung unterstützen
- in den richtigen Situationen Rückmeldung geben
- einheitliche Verfahren zur Erledigung der Arbeitsaufgaben entwickeln
- sich an die Umstände (Position) des Nutzers anpassen
- alternative Darstellungen ermöglichen, die dem Nutzer zur Verfügung stehen (Desktop-, und Android App-Präsentationslogik)

1.3 Fazit

Aufgrund der ausführlich dokumentierten Schritte des Vorgehensmodells, den angeführten Beispielen und der Vereinfachung des Lifecycles, hat sich das Team bei der weiteren Bearbeitung des Projekts für den Usability Engineering Lifecycle nach Deborah Mayhew entschieden. Besonders im Bereich des Work Reengineering wird eine erste Iteration abgewogen, die richtungsweisende Schritte für die weiteren Artefakte festigt. Sollte die Zeitplanung falsch eingeschätzt sein, können beschriebene Alternativen von Mayhew genutzt werden, um diesen Problem vorzeitig aus dem Weg zu gehen.
Durch die Bearbeitung wird nicht nur das Verständnis des Anwendungssystems für die Entwickler gesteigert, sondern auch Konzepte für die Entwicklung des Prototyps vorbereitet.

2. Modellierung anhand der Top-Down Methode

Durch das Anwenden des Top-Down Modells konnten eine passende Subdomäne gefunden werden. Hier war es wichtig darauf zu achten, dass die Subdomäne einige Eigenheiten besitzen.

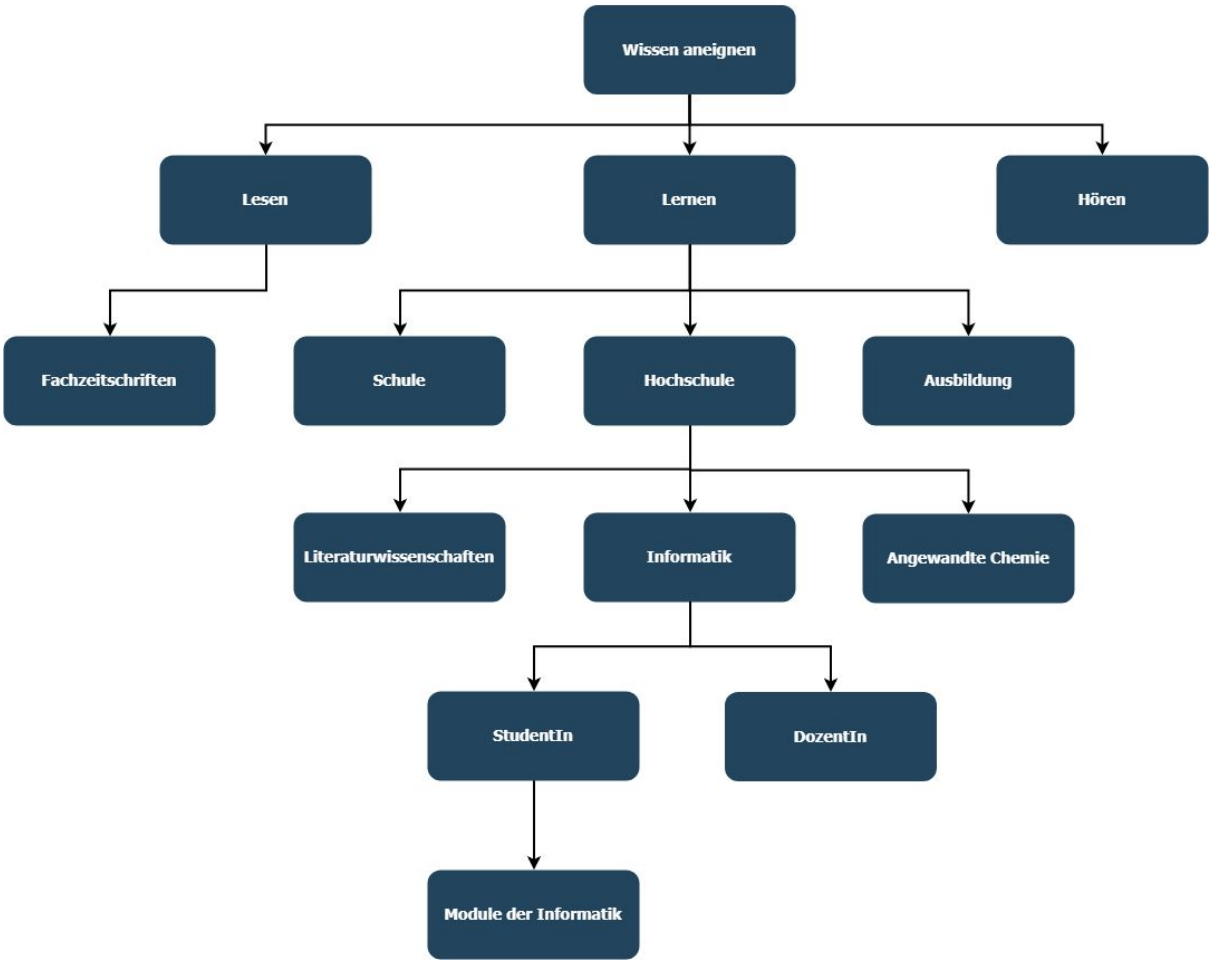


Abb. 2: Top-Down Modell Findung einer Subdomäne

Die Abbildung zeigt die genau Spezifizierung einer geeigneten Subdomäne, an der sich das Projekt orientieren wird. Im ersten Layer wurden verschiedene Arten der Aneignung von Wissen gesucht. Im Bereich lernen wurde das größte Potenzial gesehen, somit wurde der Fokus auf diese Subdomäne gelegt. Weiter wurden dann die einzelnen Lerninstitute zusammengebracht. Der zweite Layer der Hochschule mit dem dritten Layer der Informatik wurde als bestens geeignet erfunden, da beide Projektmitglieder mit diesem Fachbereich vertraut sind. Weiter soll etwas für hauptsächlich die Studenten des Studiengang gefunden werden, die Hilfe zu den spezifischen Modulen der Informatik suchen.

3. Iteration der Zielhierachie

Während des Audits zum Konzept sind Fehler in der Zielhierarchie aufgefallen. Strategische Ziele wurden zu spezifisch formuliert und im nächsten Abschnitt überarbeitet.

3.1 Strategisches Ziel

1. Der Nutzer kann das System individuell nach eigenen Vorlieben gestalten.

Taktisches Ziel

1.1 Es muss ein persönlicher Bereich implementiert werden, in dem der Nutzer seine angelegten und favorisierten Karteikartenstapel einsehen kann.

Operatives Ziel

1.1.1 Das Speichern von Karteikartenstapeln ermöglichen.

1.1.2 Das Suchen von Karteikartenstapel nach Tags, Themenbereich und Name.

Taktisches Ziel

1.2 Filterung und Sortierung der Datenbank durch Eingaben des Benutzers

Operatives Ziel

1.2.1 Am Ende der Suche sollen die passenden Karteikartenstapel angezeigt werden.

1.2.2 Alle nicht auf das Modul spezifizierten Stapel sollen ausgeblendet werden.

1.2.3 Die Filterung kann aus einer oder mehreren Optionen zusammengesetzt sein.

3.2 Strategisches Ziel

2. Die Nutzung der Anwendung soll den Erwartungen des Nutzers entsprechen.

Taktisches Ziel

2.1 Verwendung von APIs zur Berechnung und visuellen Darstellung des Standortes und der Spazier-Route.

Operatives Ziel

2.1.1 Eine Implementation einer Karte muss dem Benutzer zur direkten Verfügung stehen.

2.1.2 Das System muss mögliche Zielpunkte des Spaziergangs darstellen.

3.3 Strategisches Ziel

3. Nutzer erzielen bestmögliche Lernfortschritte.

Taktisches Ziel

3.1 Die Auswahl der Spazier-Route ist abhängig von der Größe des Karteikartenstapels und passt sich zeitlich an.

3.2 Ein effektives Lernsystem steht dem Nutzer zur Verfügung.

Operatives Ziel

3.1.1 Der Benutzer bekommt nur angemessene Routen vorgeschlagen, die in etwa dem Lernkompendium des Karteikartenstapels entsprechen.

3.2.1 Das Lernsystem funktioniert mit Karteikarten.

3.2.2 Karteikarten werden nach Priorität für Wiederholungen sortiert.

3.4 Fazit

Während der Überarbeitung wurde sich auf eine klare Unterscheidung von strategischen und taktischen Zielen konzentriert. Die Iteration dieses Artefakts ist für den weiteren Projektverlauf von hoher Wichtigkeit, um sich an den gesetzten Zielerreichungsgraden orienteren zu können.

Das erste strategische Ziel hebt die Wichtigkeit von einer funktionierenden Dateiverwaltung voraus und erleichtert den Einstieg in die Iteration der Systemarchitektur im WBA-Teil der Dokumentation. Weiterhin werden mit der Zielhierarchie Merkmale der Anwendungslogik dargestellt, die das Projektteam beachten möchte.

4. User Profiles

Im ersten Schritt des Vorgehensmodells werden User Profiles erarbeitet. Diese dienen den Entwicklern dazu wichtige Eigenschaften der Nutzergruppen auszuarbeiten und anhand dieser das User Interface Design gestalten zu können.

Für den ausgewählten Domänenraum sind folgende Nutzergruppen relevant:

- Informatikstudent
- Studenten aus anderen Studiengängen
- Schüler mittlerer Schulreife
- Professor der Informatik

Hinsichtlich projektrelevanter Merkmale werden die Nutzergruppen nun erfahrungsgemäß modelliert.

4.1 Informatikstudent

Tabelle 1: User Profile: Informatikstudent

1. Demographisch Alter Geschlecht Wohnort Sozioökonomischer Status	 18-70 Männlich/Weiblich Deutschland - Hochschulreife - Teilzeit/Vollzeit berufstätig - Studium im Informatik-Bereich
2. Berufserfahrung	Keine bis viel
3. Smartphone-Kenntnisse und -Erfahrung	Ein sehr geringer Prozentanteil besitzt kein Smartphone, dementsprechend mittelmäßig bis viel Erfahrung mit Smartphones gesammelt
4. Fachwissen	Der Nutzer studiert Informatik und hat dementsprechend nicht in jedem Fachbereich des Studiums Erfahrung
5. Spezielle Produkterfahrung	Wahrscheinlich hat der Nutzer bereits Teilfunktionalitäten des Anwendungssystems benutzt (Spazieren, Karteikarten)
6. Motivation	Informatikstudenten haben bereits das Verlangen ihr Studium abzuschließen, wofür die Anwendung Unterstützung bietet. Ebenso spielt ausreichende Bewegung eine

	wichtige Rolle für die Gesundheit des Nutzers um das Studium erfolgreich weiterführen zu können. Die Funktionen des Systems bieten sich dafür gut an.
7. Aufgaben	- Karteikartenstapel erstellen - Karteikartenstapel auswählen - Spazier-Route auswählen - Spazieren
8. Auswirkung von Fehlern	- Abschreckung des Interesses durch unverständliche Karteikarten - Wenn Spaziergang deutlich zu lang wird, kann der Nutzer gesundheitliche Schäden bekommen - Nutzer springen auf Karteikarten in Papierform zurück - Nutzer findet nicht mehr zurück
9. Häufigkeit der Nutzung	Vermehrt in Klausurphasen, sonst selten bis häufig unter der Woche

4.2 Schüler mittlerer Schulreife

Tabelle 2: User Profile: Schüler mittlerer Reife

1. Demographisch Alter Geschlecht Wohnort Sozioökonomischer Status	 16-20 Männlich/Weiblich Deutschland - mittlere Reife - Teilzeit/Vollzeit berufstätig, Ausbildung
2. Berufserfahrung	Keine bis wenig
3. Smartphone-Kenntnisse und -Erfahrung	Da viele Nutzer in dieser Altersgruppe bereits ein Smartphone besitzen, sind die Kenntnisse/Erfahrung mittel bis hoch
4. Fachwissen	Der Nutzer hat entweder kein oder mittleres Fachwissen. Kann sich entweder in der Ausbildung oder Freizeit mit Informatik beschäftigt haben.
5. Spezielle Produkterfahrung	Wahrscheinlich hat der Nutzer bereits Teilfunktionalitäten des Anwendungssystems benutzt (Spazieren, Karteikarten)
6. Motivation	Der Nutzer hat bereits Interesse an Informatik oder einem Informatikstudium und möchte Einblick in das Lernmaterial

	bekommen.
7. Aufgaben	- Karteikartenstapel auswählen - Spazier-Route auswählen - Spazieren
8. Auswirkung von Fehlern	- Abschreckung des Interesses durch unverständliche Karteikarten - Wenn Spaziergang deutlich zu lang wird, kann der Nutzer gesundheitliche Schäden bekommen - Nutzer springen auf Karteikarten in Papierform zurück - Nutzer findet nicht mehr zurück
9. Häufigkeit der Nutzung	Selten

4.3 Professor der Informatik

Tabelle 3: User Profile: Professor der Informatik

1. Demographisch Alter Geschlecht Wohnort Sozioökonomischer Status	 35-75 Männlich/Weiblich Deutschland - Vollzeit berufstätig - Erfolgreicher Abschluss des Doktors - Eingestellt an Hochschule im Bereich Informatik
2. Berufserfahrung	viel
3. Smartphone-Kenntnisse und -Erfahrung	Ein sehr geringer Prozentanteil besitzt kein Smartphone, dementsprechend mittelmäßig bis viel Erfahrung mit Smartphones gesammelt
4. Fachwissen	Der Nutzer hat in seinem Fachbereich sehr viel Erfahrung und Fachwissen gesammelt
5. Spezielle Produkterfahrung	Wahrscheinlich hat der Nutzer bereits Teilfunktionalitäten des Anwendungssystems benutzt (Spazieren, Karteikarten)
6. Motivation	Professoren möchten den Lerninhalt ihrer Veranstaltungen bestmöglich vermitteln, damit Studenten das Fach erfolgreich abschließen. Der Nutzer kann eigene, fachspezifische Karteikartenstapel für seine Studenten veröffentlichen.

7. Aufgaben	Karteikartenstapel erstellen
8. Auswirkung von Fehlern	- falsche Informationen werden an die Studenten weitergegeben -> Beschwerden
9. Häufigkeit der Nutzung	Häufig um den eigenen Lerninhalt zu veröffentlichen, danach selten (Korrekturen) bis gar nicht

Die Auswahl der Nutzergruppen bezieht sich auf primäre und sekundäre Stakeholder.
Informatikstudenten haben hierbei den größten Bedarf am System, da sie durch das Studium zur Nutzung des Lernsystems verleitet werden.
Auch Professoren, die Informatik unterrichten, haben die Möglichkeit den eigenen Lerninhalt in Form von Karteikartenstapeln zur Verfügung zu stellen. Wenn das Bedürfnis da ist, die eigenen Lerninhalte auf verschiedene Weisen zu hinterlegen, gelten auch Professoren als mögliche Nutzer des Systems.
Schüler mittlerer Reife und Studenten aus anderen Studiengängen decken sich größtenteils von der Nutzungsabsicht, weshalb nur Schüler mittlerer Reife dargestellt wurden. Diese Interessenten werden das System wahrscheinlich seltener nutzen, da es andere Informationsquellen zum Studiengang gibt.

4.4 Fazit

Mit diesem Arbeitsschritt im Vorgehensmodell mussten Nutzer spezifischer analysiert und beschrieben werden. Dadurch wurden Lücken im Konzept erkannt, die im weiteren Verlauf gefüllt werden können.

Aus den User Profiles wird erkenntlich, dass ein simples Design der Anwendung Voraussetzung für einen erfolgreichen Lernprozess der Informatikstudenten ist. Sowohl Informatikstudenten, als auch Interessenten aus anderen Lernbereichen können durch mangelndes Verständnis den Gefallen am Lernen und damit an der Anwendung verlieren.

Professoren der Informatik hingegen möchten keine falschen Informationen verbreiten und sind deshalb von einem klar verständlichen System abhängig. Sowohl Korrekturen an Karteikarten, als auch das Entfernen dieser sollte leicht gewährt sein. Für die Anwendung ist ein übersichtliches System von Vorteil.

5. Contextual Task Analysis

Durch die Herangehensweise der Contextual Task Analysis kann man die aktuellen Aufgaben in ein deskriptives Modell umwandeln. Laut Mayhew werden in diesem Schritt Hintergrundinformationen zu der Automatisierung des Arbeitsprozesses und weitere Informationen durch Interviews gesammelt, welche in einem User Task Organization Model dargestellt werden (Mayhew 1999, S.69). Daneben wird von Mayhew empfohlen, Personen zu interviewen, die in der recherchierten Domäne ansässig sind und dadurch wertvolle Informationen zur Arbeitsweise und -bereich zu erhalten. Dieses kann als Grundlage für das deskriptives Aufgabenmodell genutzt werden.

Da die Entwickler unter die vom Anwendungssystem angesprochene Nutzergruppe fallen und Zeitmangel ebenfalls ein wichtiges Risiko für das Projekt ist, wird auf Interviews verzichtet. Im Usability Engineering Lifecycle besteht ebenfalls die Möglichkeit später während der Installation Nutzererfahrungen zu sammeln.

Die Aufgabenanalyse wurde durch die Wiederbetrachtung der Anforderungen eingeleitet, um die Grenzen und Rahmen einzugrenzen.

Anschließend werden die wichtigsten Rollen im System festgelegt, indem die Stakeholder in Kombination mit den vorhergegangenen Formulierungen der User Profiles entgegengestellt werden. In diesem Projekt sind es die Lernenden und die Lehrenden - Studenten und Professoren.

Da die Komplexität des Projekts eine übersichtliche Größe annimmt, ist es möglich, die Aufgaben als Brief-Use-Case zu realisieren. Beachtung wurde neben der Beschreibung auch auf die Trigger und Probleme gelegt, die als zusätzliche Kriterien hinzugefügt wurden. Die zwei wichtigsten Task Szenarien, welche sich ebenso sehr auf unsere Alleinstellungsmerkmale fokussieren, sind hier aufgeführt.

5.1 Brief Use Cases

Tabelle 4: Brief Use Case: Lernende

Actor	Goal	Brief	Trigger	Problem
Lernender	Lerneinheit abbrechen/pausieren	Der Benutzer möchte seine angefangene Lerneinheit abbrechen	Durch Betätigung des Abbrechen-Icons auf dem angefangener Lerneinheiten Screen	Störung des Ereignis von außen
	Karteikarten stapel der	Der Benutzer möchte	Durch Betätigung des	

	Merkliste hinzufügen	Karteikarten stapel seiner Merkliste hinzufügen, damit er sie schnell wiederfinden kann	Merken Icons auf der Startseite eines Karteikartensta pels	
	Anzeige der gepeicherten Karteikartensta pel	Der Benutzer möchte sich seine gespeicherten Karteikarten stapel angucken	Durch Betätigen des Favoriten Schaltfläche in dem Navigation Menü	
	Anzeige der Lernkartenstatis tiken	Der Benutzer möchte sich die Statistiken über die gelernten Lernkarten ansehen		
	Anzeige einer Route	Der Benutzer möchte die zu gehende Route angezeigt bekommen	Dies wird bei jedem Starten einer Lerneinheit automatisch durchgeführt	Bei keiner Internetverbindu ng wird keine Route angezeigt, aber auch eine niedrige Verbindungsrat e verzögert den Aufruf stark

Tabelle 5: Brief Use Case: Lehrende

Actor	Goal	Brief	Trigger	Problem
Lehrende	Einstellen von modulspezifisch en Karteikartensta peln	Der Benutzer möchte für ein spezifisches Modul Lernkartenstape l anlegen und diese veröffentlichen	Durch Betätigen des Stapel hinzufügen Icons	

5.2 Task Szenarien

Der weitere Verlauf bezieht sich auf die Ausarbeitung von Task Szenarien, die wichtige Funktionen für das System aufgreifen und sich dabei an primäre Nutzer richten. Dank den User Profiles wurde der Informatikstudent bereits als primärer Nutzer identifiziert und in den folgenden Szenarien erfahrungsgemäß abgebildet.

5.2.1 Task: Anlegen von Karteikartenstapeln durch Karteikarten anderer Nutzer

User: Informatikstudent

Description: Als Student im 3. Semester des Studiengangs Medieninformatik hat Lars für mehrere Fächer gleichzeitig zu lernen, um die Klausur auch zu bestehen. Um den Überblick beim Lernen nicht zu verlieren möchte er Karteikartenstapel für verschiedene Fächer anlegen.

Task Flow:

1. Der Nutzer startet die Android Anwendung
2. In der Navigation klickt der Nutzer auf die Schaltfläche “Stapel erstellen”
3. Es öffnet sich eine neue Seite, in der er einen Filter durch Tags für seine Karteikarten festlegen kann
4. Die gefilterten Karteikarten werden angezeigt
5. Mit Betätigen der “Hinzufügen” Schaltfläche wird eine Karteikarte dem Karteikartenstapel hinzugefügt
6. Der Nutzer kann weitere Karteikarten seinem Stapel hinzufügen
7. Mit Betätigen der “Fertig” Schaltfläche wird der Karteikartenstapel gespeichert

Task Closure:
Da das Anlegen eines Stapels unvermittelbar und lokal gespeichert ist, hat der Nutzer keine langen Wartezeiten, sodass er entweder einen weiteren Stapel hinzufügen oder eine Lerneinheit starten kann.

Um diese Aufgabe zu unterstützen, soll das User Interface…

- … eine Möglichkeit bieten, durch ein Icon auf den Screen der Karteikartenstapel zu gelangen
- … nach Erstellen des Stapels, den Screen zur Sammlung wechseln, sodass der Nutzer sein Endergebnis direkt sieht

5.2.2 Task: Auswahl eines Karteikartenstapels für die aktuelle Lerneinheit

User: Informatikstudent

Description: Nachdem Lars nun genug Zeit am Schreibtisch mit dem Schreiben von Code verbracht hat, möchte er gerne rausgehen und dabei für die anstehenden Klausuren lernen. Die Karteikartenstapel sind bereits nach seinen Präferenzen angelegt.

Task Flow:

1. Der Nutzer startet die Android Anwendung
2. Die Schaltfläche “Jetzt loslegen” wird betätigt
3. Eine neue Seite wird geöffnet, auf der die vom Nutzer gespeicherten Karteikartenstapel zu sehen sind
4. Der Nutzer wählt einen Karteikartenstapel aus
5. Die Schaltfläche “Starten” wird betätigt

Task Closure:
Auch in diesem Task Scenario ist die Auswahl eines Stapels nicht sehr zeitintensiv und abhängig von der Anzahl der lokal gespeicherten Karteikartenstapeln.

Um diese Aufgabe zu unterstützen, sollte das User Interface…

- … dem Nutzer eine Möglichkeit bieten die Auswahl des Karteikartenstapels mit einem Kreuz-Symbol abzubrechen

5.2.3 Task: Das Hinzufügen eines Karteikartenstapels zu den eigenen Favoriten

User: Informatikstudent

Description: Durch die Vielzahl an Karteikartenstapel, welche man zu einem spezifischen Modul anlegen oder in der Suche finden kann,

Task Flow:

1. Der Nutzer startet die Android Anwendung
2. Die Schaltfläche “Jetzt loslegen” wird betätigt
3. Auswählen eines Karteikarteistapels, egal ob durch die Suchfunktion oder aus der Liste der Selbsterstellten
4. Der Nutzer wählt das Stern-Icon aus
5. Die Meldung “Karteikartenstapel wurde favorisiert” wird angezeigt

Task Closure:
Das Favorisieren von Karteikartenstapeln erleichtert dem Benutzer das wiederauffinden von Stapeln die seine Präferenzen widerspiegeln

Um diese Aufgabe zu unterstützen, sollte das User Interface…

- … dem Nutzer eine Möglichkeit bieten Karteikartenstapeln nur durch die Betätigung eines Icons in eine Favoritenliste speichern zu können.

5.2.4 Liste von weiteren möglichen Task Scenarios

Liste von User Tasks für Informatikstudenten

- Erzeugen von Karteikartenstapeln
- Auflistung der erzeugten Karteikartenstapel
- Auflistung der favorisierten Karteikartenstapel
- Auswahl einer Route, nachdem der Karteikartenstapel ausgewählt wurde
- Suchen von Karteikartenstapeln
- Anmeldung und Registrierung
- Suchen und Filterung von Karteikartenstapeln

Liste von User Tasks für Dozenten

- Erzeugen von Karteikartenstapeln
- Anmeldung und Registrierung

Für die fortlaufende Entwicklung ist es wichtig, dass das System dem Nutzer Aufgabenangemessenheit bietet. User Tasks, wie das Anlegen von Karteikartenstapeln oder die Auswahl einer geeigneten Spazierroute haben hierbei eine hohe Priorität, um die Gebrauchstauglichkeit zu erhöhen.

5.3 Current User Task Organization Model

Im letzten Schritt der Contextual Task Analysis werden User Task Organization Models vorgeschrieben, die dazu verhelfen alle grundlegenden Aufgaben der Nutzer zu identifizieren und modellieren.

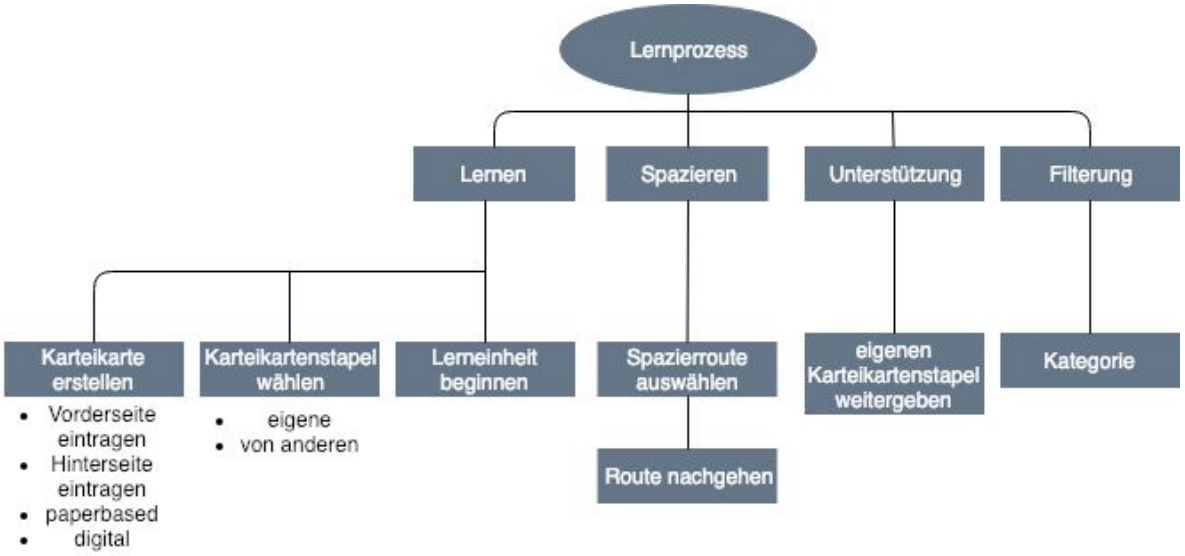


Abb. 3: Current User Task Organization Model Lernprozess

Während der Bearbeitung des Modells (siehe Abb. 3) wurde nach dem Top-Down Modell vorgegangen. Hiermit wird das Zusammenspiel der Funktionalitäten auf mehreren Ebenen dargestellt.
Da die Komplexität der Anwendung nicht hoch ist, wurde ein Shortcut des Vorgehensmodells angewandt: Der Fokus liegt auf der primären Nutzergruppe, also Informatikstudenten, und stellt ein Modell der Verwaltung von Karteikarten und des Spazierengehens dar.
Ein User Task Organization Model für Professoren ist somit inbegriffen und muss weiterhin nicht dargestellt werden.

Der Text in den Boxen repräsentiert grundlegende User Tasks und wird durch zusätzliche Ebenen und Beschreibungen der Attribute genauer definiert. Während der Bearbeitung wurde das Modell bewusst projektspezifisch dargestellt, um nicht zu viele Alternativen und damit ein Überfluss an unnötigen Informationen darzustellen.

Für den weiteren Verlauf des Vorgehensmodells bietet dieser deskriptive Zustand eine Orientierungsmöglichkeit, um Erkenntnisse von den Umstände des Nutzers nutzen zu können.

6. Usability Goals

Im Rahmen der Gebrauchstauglichkeit helfen Usability Goals dazu Ziele für den Entwicklungsprozess zu finden. Die zuvor dargestellten Ergebnisse der User Profiles und die darauf bauende Task Analysis werden hierbei berücksichtigt.

6.1 Qualitative Usability Goals

In den folgenden Abschnitten werden qualitative Usability Goals aufgelistet, die zielführend für die Gestaltung sind. Das Projektteam ist hierbei auf Deborah Mayhew's Prioritäten (Mayhew 1999, S.136) eingegangen, welche folgende Bedeutung haben:

- 1 = Wird für die Veröffentlichung benötigt
- 2 = Wichtig, wenn die Kosten nicht zu hoch sind oder zu viel Zeit verlangt wird
- 3 = Wünschenswert, wenn die Kosten niedrig sind

Die nachfolgenden Ziele entstehen durch eine weniger intensive Recherche, da das Projekt bereits als weniger umfassend eingestuft wurde.

6.1.1 Ziel 1: Die grafische Gestaltung der Anwendung soll für den Nutzer leicht verständlich sein

Priorität: 1

Damit das Prinzip der App für Erstanwender selbsterklärend ist, sollte das System nur die nötigsten grafischen Elemente darstellen. Dem Nutzer sollte die Funktionsweise der Anwendung bei bloßer Betrachtung klar werden.

6.1.2 Ziel 2: Der Nutzer soll eigene Karteikarten schnell anlegen können, um den Lernprozess zu optimieren

Priorität: 1

Für Karteikarten ist es wichtig, dass sowohl die Vorder- als auch Rückseite beschriftet werden soll. Der Nutzer muss also einen übersichtlichen und schnellen Zugriff auf die Bearbeitung der Karteikarte haben, damit die Vorbereitung auf den Lernprozess nicht zu lange dauert. Weiterhin sollen Tags schnell gesetzt werden können, die eine Filterung der Karteikarten ermöglichen.

6.1.3 Ziel 3: Der Nutzer soll eigene Karteikartenstapel einfach und schnell erstellen können

Priorität: 1

Für die simple Anwendung spricht ebenfalls ein hilfsbereites System, welches dem Nutzer mit nachvollziehbaren Schritten die Interaktion erleichtert. Damit das Erstellen von Karteikartenstapeln den Prozess nicht großartig behindert, kann das Design hier mit klaren Anweisungen Unterstützung bieten.

6.1.4 Ziel 4: Die Nutzerschnittstelle sollte selbsterklärend sein, damit Spaziergänge sofort mit einer Lerneinheit gestartet werden können

Priorität: 1

Damit der Nutzer Spazierrouten vorgeschlagen bekommt, ist die Auswahl eines Karteikartenstapels nötig. Karteikartenstapel sollten einer geordneten Hierarchie angehören und logisch in den gesetzten Tags abrufbar sein, damit die Vorbereitung für einen Spaziergang nicht zu lange dauert. Weiterhin sollten sinnvolle Rückmeldungen vom System angegeben werden, damit dem Nutzer die Dauer der Lerneinheit bewusst wird.

6.1.5 Ziel 5: Das System sollte die Nutzer authentifizieren können

Priorität: 2

Der aktuelle Projektverlauf setzt eine Nutzung des Systems für Nicht-Informatikstudenten voraus. Da einige Vorlesungsinhalte für Außenstehende nicht zugänglich sein sollten, müsste das Anwendungssystem eine Möglichkeit bieten Studenten erkennen zu können. Auch für die Auswahl eigener Karteikartenstapel sollte es dem Nutzer möglich sein ein eigenes Konto anzulegen.
Für den Prototypen wurde diesem Ziel keine hohe Priorität gegeben, da eine Veranschaulichung der Funktionen auch ohne Authentifizierung möglich ist.

6.1.6 Ziel 6: Die Spazier-Route sollte schnellstmöglich angezeigt werden

Priorität: 2

Wenn der Nutzer einen Karteikartenstapel gewählt hat, sollten die möglichen Routen nicht zu lange brauchen um angezeigt zu werden. Zu lange Wartezeiten könnten dazu führen, dass der Nutzer eine Fehlermeldung erwartet und die Lerneinheit frühzeitig abbricht. Vermeidbar wäre dies durch eine Kenntnissetzung über zu lange Wartezeiten für den Nutzer.

6.1.7 Ziel 7: Das Design sollte die Nutzer dazu animieren das System öfter zu nutzen

Priorität: 2

Die Zufriedenheit des Nutzers hat einen hohen Stellenwert und sollte im Design angesprochen werden. Das System sollte nicht nur mit seinen Funktionen überzeugen, sondern auch auf logische Komponenten bauen, die die Benutzung angenehmer macht. So wär es für einen Nutzer nicht nur möglich die gleiche Spazierroute zurückzugehen, sondern auch eine andere Route für den Heimweg auszuwählen und neue Orte kennenzulernen. Weiterhin ist es für den Nutzer hilfreich die Karteikarten priorisiert angezeigt zu bekommen, damit schwierige und selten aufgerufene Karteikarten eher gelernt werden.
Die Priorität dieses Ziels ist gering, da das System sehr wahrscheinlich ohnehin oft benutzt wird, wenn der Bedarf vorhanden ist (zum Beispiel in einer Klausurphase).

6.1.8 Ziel 8: Die vom Professor eingereichten Karteikartenstapel sollten markiert werden

Priorität: 3

Da die Anzahl an Karteikartenstapeln abhängig von der Anzahl der Nutzer ist, sollten diese nicht nur filterbar sein, sondern auch markiert werden, wenn ein Professor der Informatik einen Stapel erstellt hat.
Die Priorität ist hier nicht hoch, da das Nutzen der Anwendung erst von einem Professor der Informatik akzeptiert werden muss und nicht notwendig für ein funktionierendes System für Informatikstudenten ist.

6.2 Quantitative Usability Goals

Die zuvor ausgewählten qualitativen Usability Goals wurden von dem Projektteam im weiteren Verlauf anhand der Prioritäten ausgewertet und kategorisiert. Im Konzept wurden bereits Anforderungen für die Funktionalitäten der Anwendung aufgestellt. Mit den von Deborah Mayhew veranschaulichten Vorlagen (Mayhew 1999, S.142) werden im weiteren Verlauf ausgewählte Usability Goals hervorgehoben.

Usability Goals

Goal #: 1

Task: Karteikartenstapel anlegen

Operational Definitions:

Expert: 3. Versuch
Novice: 1.-2. Versuch
Learn: 1 = sehr niedrig
Satisfaction: 4 = neutral, 7 = sehr hoch

Priority Definitions

1= nötig
2= Wichtig, wenn Zeit/Kosten nicht hoch
3= Wünschenswert

Ease-of-Learning Goals

Priority	Measure	Goal
	N-Time	
1	N-Trials	3
	N-Errors	

Ease-of-Use Goals

Priority	Measure	Goal
	E-Time	
1	E-Errors	0

Satisfaction Goals

Priority	Measure	Goal
2	Expert	6
3	Novice	4

Abb. 4: Usability Goal 1

Das erste Usability Goal (siehe Abb. 4) befasst sich mit dem zweiten Punkt der funktionalen Anforderungen im Konzept. Neulinge sollten für das Anlegen eines Karteikartenstapels drei Versuche benötigen um als Experte bezeichnet zu werden. Die Zeitkomponente wurde hierbei bewusst rausgelassen, da das Erstellen eines Karteikartenstapels zeitlich variieren kann, abhängig von der Anzahl der ausgewählten Karteikarten. Das Satisfaction Goal wird erreicht, wenn die Zufriedenheit der Experten 6 erreicht und bei Neulingen 4, da Neulinge sich erst mit dem System vertraut machen müssen, bevor sie mehr Gefallen finden können.

Usability Goals

Goal #: 2

Task: Spazier-Routen Auswahl

Operational Definitions:

Expert: 2. Versuch
Novice: 1. Versuch
Learn: 1 = sehr niedrig
Satisfaction: 4 = neutral, 7 = sehr hoch

Priority Definitions

1= nötig
2= Wichtig, wenn Zeit/Kosten nicht hoch
3= Wünschenswert

Ease-of-Learning Goals

Priority	Measure	Goal
1	N-Time	10s
1	N-Trials	2
	N-Errors	

Ease-of-Use Goals

Priority	Measure	Goal
2	E-Time	5s
1	E-Errors	0

Satisfaction Goals

Priority	Measure	Goal
2	Expert	6
3	Novice	6

Abb. 5: Usability Goal 2

Dieses Usability Goal (siehe Abb. 5) befasst sich mit den Punkten 3, 4 und 11 aus den funktionalen Anforderungen. Wichtig ist hierbei, dass die Wahl von Spazierrouten als Experte halb so lang dauert wie bei Neulingen und der Experten-Rang bereits nach 2 Versuchen erreicht wird. Da die Zufriedenheit unter anderem abhängig von der Google Maps API ist und die Umsetzung dadurch erleichtert wird, ist die Priorität bei den Satisfaction Goals nicht so hoch eingestuft.

Usability Goals

Goal #: 3

Task: Profil einloggen

Operational Definitions:

Expert: 2. Versuch
Novice: 1. Versuch
Learn: 1 = sehr niedrig
Satisfaction: 4 = neutral, 7 = sehr hoch

Priority Definitions

1= nötig
2= Wichtig, wenn Zeit/Kosten nicht hoch
3= Wünschenswert

Ease-of-Learning Goals

Priority	Measure	Goal
1	N-Time	60s
1	N-Trials	1
1	N-Errors	0

Ease-of-Use Goals

Priority	Measure	Goal
1	E-Time	10s
1	E-Errors	0

Satisfaction Goals

Priority	Measure	Goal
2	Expert	6
3	Novice	6

Abb. 6: Usability Goal 3

Im ersten Punkt (siehe Abb. 6) der funktionalen Anforderungen wird das Login Verfahren hervorgehoben. Während Neulinge erst eine Registrierung abschließen müssen, sollen

Experten ab dem 2. Login Versuch eine schnellere Anmeldung von bis zu 10 Sekunden Länge ermöglicht werden. Da ein Login für den weiteren Verlauf der Anwendung leicht implementierbar, aber auch nötig ist, wurden die Satisfaction Goals hoch eingestuft und die Priorität niedriger.

6.3 Fazit

Die Usability Goals haben verdeutlicht, welche Aspekte besonders wichtig für den weiteren Verlauf des Prototyps ist. Eine simple und selbsterklärende Gestaltung, sowie effektive Funktionen bilden hierfür eine stabile Grundlage. Dank den quantitativen Usability Goals ist ein Maßstab für weitere Gestaltungen des Anwendungssystems gegeben.

7. Platform Capabilities and Constraints

User Interface Designer brauchen Möglichkeiten um feststellen zu können, ob Einschränkungen und Änderungen von Implementationsansätzen für spezielle Plattformen als Soft- und Hardware berücksichtigt werden müssen.
Die Umsetzung unseres Systems erfolgt im Front End als Android Java Anwendung und Desktopanwendung, basierend auf Javascript, HTML und CSS, sowie im Back End auf Javascript mit NodeJS.
In den folgenden Tabellen werden Möglichkeiten und Einschränkungen für die jeweiligen Anwendungen dargestellt.

Tabelle 6: Plattform: Desktopanwendung

Eigenschaften	Möglich	Möglich mit zusätzlichem Aufwand	Nicht möglich
Betriebssystem	Plattformübergreifend: 10,8,7, sowie Mac und Linux		
Display Größe	12 Zoll und größer		
Eingabe-Gerät	Physische Tastatur, Bildschirmtastatur, Maus		
Internetverbindung	Verbindung über LAN und WLAN		
Farben	Alle beliebigen Farben des 32-Bit Raums		
Spezial-Effekte	3D, Video, Audio		
GUI Werkzeuge	CSS HTML	Individuelles Design	
Energieversorgung	Netzbetrieb, Akkulaufzeit bei Laptops		
Multitasking	x		
Bit-Mapped-Display	x		
Windowing	Ja, da im Internetbrowser		

Tabelle 7: Plattform: Android Anwendung

Eigenschaften	Möglich	Möglich mit zusätzlichem Aufwand	Nicht möglich
Betriebssystem	4.0.3 oder höher		Ältere Versionen
Display Größe	4 Zoll und größer		Kleinere Auflösungen
Eingabe-Gerät	Virtuelle Tastatur, Touchscreen, physikalische Tastatur und Maus		
Internetverbindung	WLAN, mobiles Internet, sowie LAN		
Farben	Alle beliebigen Farben des 32-Bit Raums		
Spezial-Effekte	3D, Video, Audio		
GUI Werkzeuge	Android GUI	Individuelles Design	
Energieversorgung	Akkulaufzeit, oder Netzbetrieb		
Multitasking	x		
Bit-Mapped-Display	x		
Windowing	ab Android 8.0 möglich		

Zwar bietet das Vorgehensmodell die Option dieses Kapitel zu überspringen, wenn die Umstände von Soft- und Hardware bereits bekannt sind, jedoch haben sich die Entwickler dazu entschlossen einen Überblick von wichtigen Bedingungen darzustellen.
Die tabellarische Auflistung wurde direkt aus Mayhews Buch übernommen (Mayhew 1999, S. 157) und mit eigenen Eigenschaften erweitert. Außerdem legt die Tabelle einen leicht ablesbaren Rahmen fest, auf der die Implementation beruhen wird.

8. Gestaltungsprinzipien

Als Grundlage für die Gestaltung können weitere Gestaltungsprinzipien recherchiert werden. Da das Anwendungssystem sowohl als Desktop als auch Android Anwendung realisiert werden kann, wurden Prinzipien gesucht, die für beide Endgeräte anwendbar sind.

8.1 “Die Navigation in UWP-Apps basiert auf einem flexiblen Modell aus Navigationsstrukturen, Navigationselementen und Funktionen auf Systemebene.”[2]

Mit diesem Gestaltungsprinzip von Microsoft ist auf die Flexibilität und Erreichbarkeit von Unterseiten innerhalb der Anwendung gezielt. Da der Prototyp mehrere Unterseiten haben wird, sind einheitliche Navigationselemente fundamental für die Gestaltung. Die Hierarchie der Unterseiten sollte demnach auch selbstbeschreibend sein.

8.2 Cross Platform

“Material Design maintains the same UI across platforms, using shared components across Android, iOS, Flutter, and the web.”[3]

Eine einheitliche Nutzerschnittstelle erhöht ebenfalls den Lernprozess des Nutzers. Für Karteikarten, Karteikartenstapel und andere Elemente der Anwendung können ähnliche Symbole benutzt werden.

8.3 Bold, graphical, intentional

“Material Design is guided by print design methods — typography, grids, space, scale, color, and imagery — to create hierarchy, meaning, and focus that immerse viewers in the experience.”[3]

Um diesen Gestaltungsprinzip gerecht zu werden, sollte ein einheitliches Schema für die oben genannten Faktoren angewandt werden. Die Zufriedenheit des Nutzers hat eine hohe Priorität und wirkt sich auf die Häufigkeit des Nutzens aus.

8.4 Only show what I need when I need it

„People get overwhelmed when they see too much at once. Break tasks and information into small, digestible chunks. Hide options that aren't essential at the moment, and teach people as they go."[4]

Der Nutzer soll nicht mit Informationen überladen werden. Hilfreich ist hierbei das Prinzip der Paginierung, wodurch sehr große Ressourcen auf kleinere Projektionen runtergebrochen werden. Besitzt ein Karteikartenstapel sehr viele Karteikarten, sollte dementsprechend nur ein Teil der Karteikarten übersichtlich und mit den nötigen Informationen dargestellt werden.

Für die Bearbeitung hat das Team eine Abkürzung genommen, die die Anzahl der Ressourcen auf wenige limitiert. Einige recherchierte Quellen bieten nützliche Informationen für die Gestaltung der Anwendung, trotzdem verdeutlichen Artefakte des Konzepts und der Dokumentation bereits ausreichende Informationen über die Gestaltung.

9. Work Reengineering

Um einen genaueren Bezug zu den Nutzern und deren Aufgaben zu identifizieren ist laut Mayhew ein Work Reengineering Prozess notwendig.

Mayhew zitiert hierbei Button und Dourish “You want to ‘... align system design not so much with the details of specific working practices, as with the details of the means by which such working practices arise and are constituted.’”(Mayhew 1999, S.171).
Damit ist gemeint, dass die Ursache für Arbeitsprozesse hinterfragt und beim Design miteinbezogen werden soll. Es soll hervorgehoben werden welche Schritte unternommen werden müssen, damit ein Informatikstudent effizient die Lerneinheit zu starten.

In den nachfolgenden Abschnitten werden Use Cases, Aufgabenmodelle und das User Task Organization Model auf die genannten Vorteile eines Work Reengineering Prozesses überarbeitet. Dadurch wird die Effizienz von Automatisierungen und der Einbezug von menschlicher Kognition für die Aufgabengestaltung hervorgehoben.

9.1 Use Cases

Da in der Task Analysis nur Brief Use Cases erarbeitet wurden, hat sich das Team an den erarbeiteten User Profiles und Task Szenarien orientiert um detaillierte Use Cases modellieren zu können.
Für die nachfolgenden Use Cases wurde sich auf die Verwaltung der Karteikartenstapel und Google Maps API Interaktion konzentriert, da diese Funktionen das Alleinstellungsmerkmal ausmachen und von hoher Wichtigkeit für den Prototypen sind.

Tabelle 8: Use Case 1: Karteikartenstapel auswählen

Use Case 1: Der User des Android Clients wählt für seine Lerneinheit einen Karteikartenstapel aus	
Primary Actor	User
Goal of Context	User möchte Lerneinheit auswählen
Scope	calmRunter – Android Client
Preconditions	Applikation ist vorinstalliert User is eingeloggt
Success End Condition	Die ausgeführte Lerneinheit wurde erfolgreich gestartet
Failed End Condition	Die Lerneinheit konnte nicht abgebrochen werden
Trigger	Applikation wird gestartet.

Normal Flow	1	Schaltfläche “Lernen” wählen
	2	Karteikartenstapel auswählen
	3	Schaltfläche “Jetzt loslegen” wählen

Tabelle 9: Use Case 2: Lerneinheit abbrechen

Use Case 2: Der Nutzer des Android Clients bricht seine aktuelle Lerneinheit ab		
Primary Actor	User	
Goal of Context	User möchte gestartete Lerneinheit abbrechen	
Scope	calmRunter – Android Client	
Preconditions	Applikation ist vorinstalliert User ist eingeloggt	
Success End Condition	Die ausgeführte Lerneinheit wurde erfolgreich beendet	
Failed End Condition	Die Lerneinheit konnte nicht abgebrochen werden	
Trigger	Schaltfläche “Jetzt loslegen” gewählt	
Normal Flow	1	Die Navigation öffnen mit Betätigung des Navigations Icons
	2	Schaltfläche “Jetzt abbrechen” wählen
	3	Ausgabe der Meldung, dass die Aktion abgebrochen wurde

Tabelle 10: Use Case: Karteikartenstapel der Merkliste hinzufügen

Use Case 3: Der Nutzer des Android Clients fügt einen Karteikartenstapel seiner Merkliste hinzu	
Primary Actor	User
Goal of Context	User möchte einen Karteikartenstapel zu seiner Merkliste hinzufügen
Scope	calmRunter – Android Client
Preconditions	Applikation ist vorinstalliert User ist eingeloggt
Success End Condition	Der Karteikartenstapel wurde erfolgreich der Merkliste hinzugefügt

Failed End Condition	Der Karteikartenstapel wurde der Merkliste nicht hinzugefügt	
Trigger	Applikation wird gestartet.	
Normal Flow	1	Schaltfläche “Lernen” auswählen
	2	Nutzer wählt einen Karteikartenstapel aus
	3	Hinzufügen zur Merkliste durch das Betätigen das Stern Icon
	4	Das Stern Icon verfärbt sich als Indikator des Hinzufügens

Tabelle 11: Use Case: Merkliste ansehen

Use Case 4: Der Nutzer möchte sich seine gespeicherten Karteikartenstapel ansehen		
Primary Actor	User	
Goal of Context	User sieht alle gespeicherten Karteikartenstapel	
Scope	calmRunter – Android Client calmRunter – Desktop Client	
Preconditions	Applikation ist vorinstalliert User ist eingeloggt	
Success End Condition	Eine Liste der gespeicherten Stapel wird dem User angezeigt	
Failed End Condition	Eine leere Liste wird dem Benutzer angezeigt	
Trigger	Applikation wird gestartet.	
Normal Flow	1	Navigation öffnen
	2	Die Schaltfläche “Karteikartenstapel” betätigen
	3	Aufrufen des Karteikartenstapel-Screen

Tabelle 12: Use Case:Statistiken ansehen

Use Case 5: Der Nutzer möchte eine Spazier-Route auswählen		
Primary Actor	User	
Goal of Context	Die Spazier-Route wird dem User angezeigt	
Scope	calmRunter – Android Client	
Preconditions	Applikation ist vorinstalliert GPS ist verfügbar User ist eingeloggt	
Success End Condition	Dem Nutzer wird die aktuelle Lerneinheit und Spazier-Route angezeigt	
Failed End Condition	Eine Lerneinheit wird nicht gestartet	
Trigger	Schaltfläche “Jetzt loslegen” ausgewählt	
Normal Flow	1	Markierung betätigen
	2	Schaltfläche “Route wählen” auswählen

Tabelle 13: Use Case:Statistiken ansehen

Use Case 6: Der Nutzer möchte fremde verfügbare Karteikartenstapel sehen		
Primary Actor	User	
Goal of Context	Der Nutzer möchte andere Karteikartenstapel zum Lernen ausprobieren	
Scope	calmRunter – Android Client	
Preconditions	Applikation ist vorinstalliert User ist eingeloggt	
Success End Condition	Alle verfügbaren Karteikartenstapel sind sichtbar	
Failed End Condition	Der Nutzer sieht keine fremden Karteikartenstapel	
Trigger	Applikation wird gestartet.	
Normal Flow	1	Schaltfläche “Mehr Karteikartenstapel” auswählen
	2	Fachbereich auswählen

Tabelle 14: Use Case:Statistiken ansehen

Use Case 7: Anlegen eines neuen Karteikartenstapels		
Primary Actor	User	
Goal of Context	Der Nutzer einen neuen Karteikartenstapel anlegen	
Scope	calmRunter – Android Client calmRunter - Desktop Client	
Preconditions	Applikation ist vorinstalliert User ist eingeloggt	
Success End Condition	Ein Karteikartenstapel wurde erfolgreich angelegt	
Failed End Condition	Es konnte kein neuer Karteikartenstapel angelegt werden	
Normal Flow	1	Schaltfläche “Karteikartenstapel” auswählen
	2	Schaltfläche “Meine Karteikartenstapel” auswählen
	3	Schaltfläche “Neuer Stapel” auswählen

9.2 Iteration Task Scenarios

Für das Überarbeiten der Task Scenarios wurde die Hierarchical Task Analysis aus dem Konzept betrachtet. Es ist aufgefallen, dass einige Funktionen des Anwendungssystems fehlinterpretiert wurden und in folgenden Diagrammen verbessert dargestellt werden können.

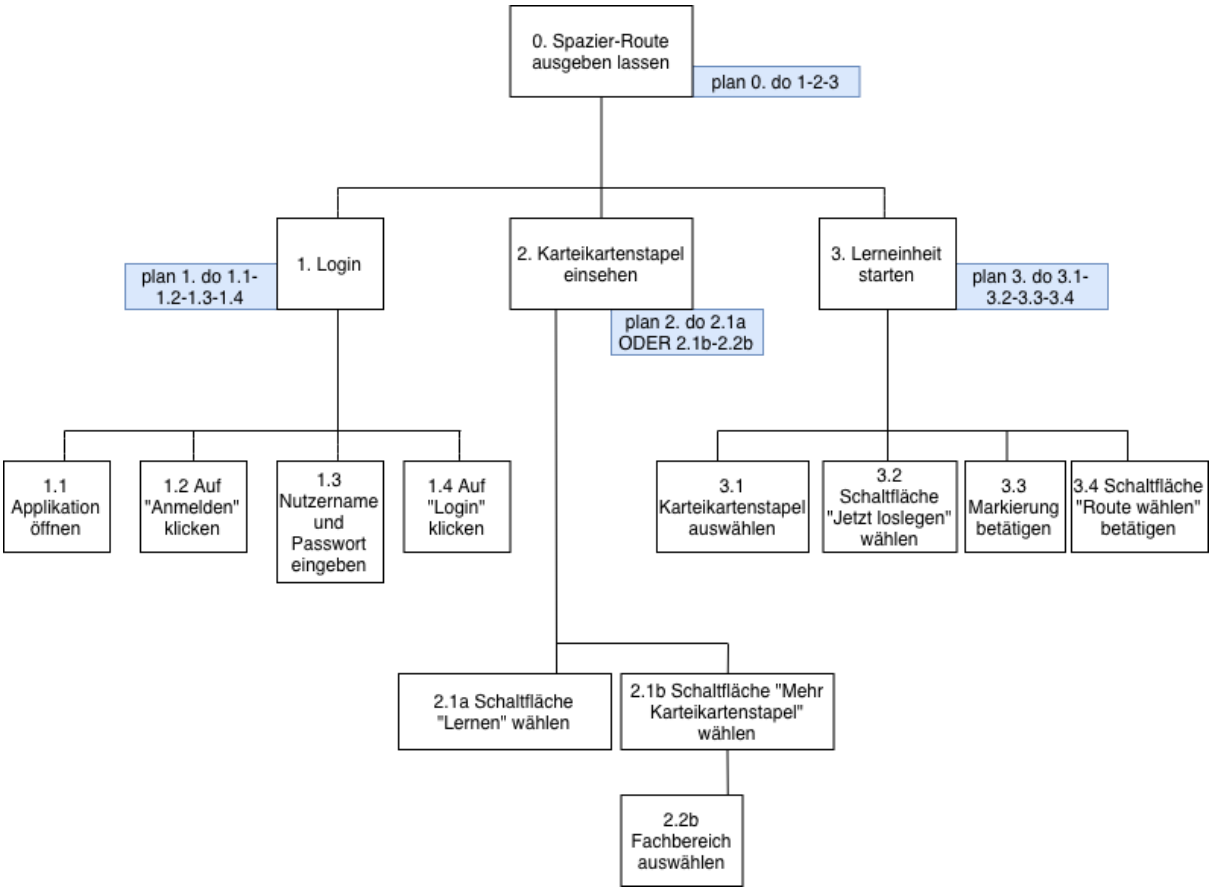


Abb. 7: Task Scenario Spazier-Route

Bei diesem Szenario (siehe Abb. 7) wurde das Einsehen der Karteikartenstapel an die erarbeiteten Artefakte angepasst. Dem Nutzer ist es nun möglich entweder eigene Karteikartenstapel zu nutzen oder unter “Mehr Karteikartenstapel” mit den Karteikartenstapeln von anderen Nutzern zu lernen.

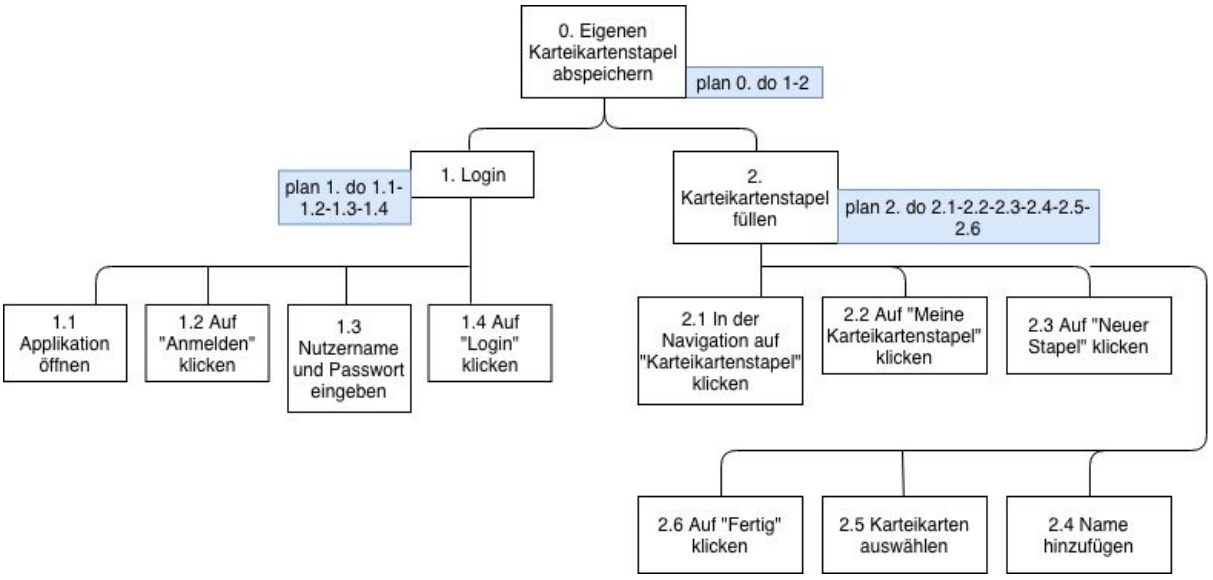


Abb. 8: Task Scenario Abspeichern

Wesentliche Änderungen wurden in diesem Szenario (siehe Abb. 8) durch die Contextual Task Analysis ausgearbeitet. Die Speicherung von Karteikartenstapeln kann sich sowohl auf eigene Karteikarten, als auch auf die Karteikartenstapel von anderen beziehen und sollte somit unterschiedlich behandelt werden. Da das Speichern eigener Karteikartenstapel mehr Arbeitsschritte benötigt und die Interaktion mit anderen Karteikartenstapeln bereits als Task Scenario grob modelliert wurde, wurde auf das letztere Modell verzichtet.

9.3 Reengineered Task Organization Model

Im letzten Schritt wird das Current User Task Model evaluiert, um mögliche Schwachpunkte in der Konzeption zu finden. Ziel ist es hierbei nicht jede kleinste Aufgabeneinheit zu modellieren oder alle möglichen Nutzergruppen darzustellen, sondern eine reduzierte Form der Arbeitsprozesse für mehr Effizienz zu finden. Wie bereits im Current User Task Model werden die Arbeitsprozesse der primären Nutzergruppe dargestellt.

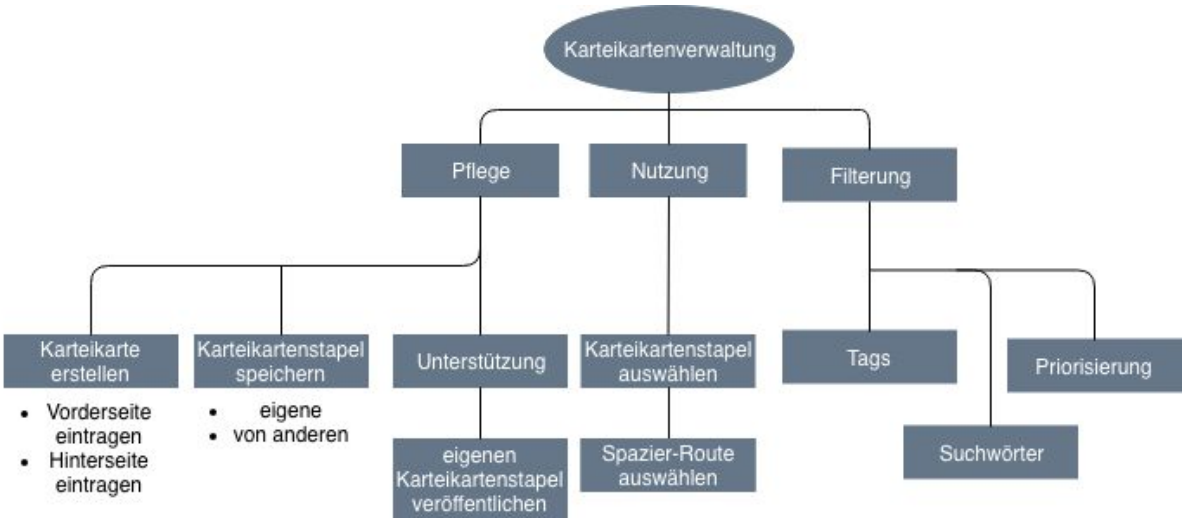


Abb. 9: Reengineered Task Organization Model

Durch den Reengineering Prozess (siehe Abb. 9) wurden die höheren Ebenen der Hierarchie erneut aufgearbeitet. Die Pflege der Karteikartenverwaltung beinhaltet nun ebenfalls die Veröffentlichung der eigenen Karteikartenstapel, um andere Nutzer zu unterstützen. Diese Funktion ging aus dem 6. Use Case des Work Reengineering aus Kapitel 5 hervor.
Ein weiterer wichtiger Punkt ist die Filterung, welche durch eine Priorisierung erweitert wurde. Durch das 8. Ziel der Usability Goals aus Kapitel 6 wurde dieser Aspekt ausgearbeitet und soll im System für eine angenehmere Nutzung sorgen.

9.4 Fazit

Abschließend hat der Work Reengineering Prozess eine detaillierte Übersicht geschaffen, die den weiteren Projektverlauf unterstützt. Da das Team in der konzeptbasierten Aufgabenanalyse nur Brief Use Cases veranschaulicht hat, konnten die in diesem Kapitel veranschaulichten Use Cases mehr Klarheit schaffen. Sowohl für das Hierarchische Aufgaben Modell, als auch das User Task Organization Model konnten Wege durch andere Ausarbeitungen gefunden werden, die den Gestaltungsprozess fortan vereinfachen.

10. Konzeptuelles Modell

Mayhew beschreibt den anstehenden Gestaltungsprozess in drei Ebenen: das konzeptuelle Modell Design, die Screen Design Standards und das Detailed User Interface Design.

Basierend auf den vorläufigen Ausarbeitungen steht im nächsten Schritt das konzeptuelle Modell an. Im Fokus liegt hierbei das Reengineered Task Organization Model, welches laut der Autorin den Grundbaustein für eine hochgradige und regelbasierte Nutzerschnittstelle bietet (Mayhew 1999, S.189).

10.1 Produkt- oder prozessorientiert

Für den weiteren Verlauf des Design Prozesses muss zuerst abgewogen werden, ob das konzeptuelle Modell produkt- oder prozessorientiert gestaltet werden soll.

Die produktorientierte Gestaltung bietet Hilfestellung für arbeitsorientierte Anwendungen, die individuelle Produkte erstellen, benennen und speichern können. Als Beispiel wurden hier Office Anwendungen wie Word, Excel oder PowerPoint genannt, die das Erstellen und Bearbeiten von Dokumenten ermöglichen.
Auf der anderen Seite bietet eine prozessorientierte Gestaltung ein Modell für Anwendungen, die kein klar identifizierbares Produkt verfolgt. Ebenfalls konzentriert sich die Gestaltung auf den Arbeitsprozess selber und verfolgt selbst definierte Modelle, da keine konkreten Produkte angestrebt werden und der Entwickler sich somit an keine Schablone halten kann. Nutzer können über prozessorientierte Anwendungen persönliche Daten speichern und empfangen, jedoch überwiegend so, dass jeder Nutzer auf die gleichen Daten Zugriff hat.

Die Anwendung wird prozessorientiert eingeordnet, da Karteikartenstapel gespeichert und unter den Nutzern geteilt werden können. Weiterhin steht kein bestimmtes Produkt im Fokus, sondern die effektive Gestaltung des Nutzungsprozesses, damit die Bedürfnisse und Erwartungen der primären Nutzergruppe angesprochen wird.

10.2 Eindeutige Prozess-Identifizierung

Da sich für eine prozessorientierte Vorgehensweise entschieden wurde, reicht bereits das Reengineered Task Organization Model um einen Überblick der Prozesse zu bekommen. Dadurch kann dieser Schritt übersprungen werden.

10.3 Design Präsentation Regeln

Zunächst werden Repräsentation der einzelnen Ebenen des Reengineered Task Organization Model festgelegt. Es wird als Ziel gesetzt eine visuelle Modellierung der angegebenen Prozesse auszuarbeiten.

Das Anwendungssystem hat, wie bereits im ersten Modell der Systemarchitektur veranschaulicht, zwei verschiedene Präsentationslogiken: Die Ansichten der mobilen Anwendung und Desktopanwendung. Deborah Mayhew gibt eine Hierarchie mit 3 Ebenen vor, an denen sich orientiert wurde. Während für die Android App keine Vorgaben gegeben werden, lässt sich die Desktopanwendung stark an der Vorlage von Mayhew orientieren. (Mayhew 1999, S.195)

10.3.1 Desktopanwendung

Die erste Ebene wird für die Desktopanwendung dementsprechend als Tabs dargestellt, wodurch sich gruppierte Selektionen der Bereiche, wie Karteikartenstapel oder Merklisten, ermöglichen.
Für die zweite Ebene ist ein Untermenü innerhalb der Tabs vorgesehen, um die Hierarchie genauer darzustellen. So sollen Karteikarten und Karteikartenstapel unter dem Tab Karteikarten verfügbar und auswählbar sein, während die Merkliste eine Auflistung von gespeicherten Karteikarten und Karteikartenstapeln enthält.
Auf der letzten Ebene werden weitere Prozesse als Pull-Down von dem Untermenü veranschaulicht. Realisiert wird hiermit das Erstellen, Bearbeiten und Entfernen verschiedener Komponenten.
Weitere Elemente, wie das Schließen und Minimieren des Fensters sind in Form von Symbolen dargestellt.

10.3.2 Mobile Anwendung

Die Präsentationslogik der Android Applikation fokussiert sich auf den Informatikstudenten und listet somit mehr Funktionen als die Desktopanwendung auf.
Auf der ersten Ebene soll weiterhin eine gruppierte Selektion von Prozessen möglich sein. Der Startbildschirm soll die Auswahl des Karteikartenstapels und ebenfalls den Spaziergang veranschaulichen. Ebenfalls wurde sich dazu entschlossen ein Seitenmenü einzubauen, welches durch ein Menü-Symbol aufrufbar ist. Karteikartenstapel und einzelne Karteikarten sind dadurch zugänglich.
Auf der zweiten Ebene befinden sich Unterteilungen zwischen eigenen und fremden Karteikarten.
Die dritte Ebene zeigt erneut das Erstellen, Bearbeiten und Entfernen der genannten Elemente. Zur Orientierung dient eine Überschrift des geöffneten Tabs. Durch das Aufrufen der höheren Ebenen werden neue Screens veranschaulicht, die durch ein Zurück-Symbol verlassen werden können.

10.4 Ausarbeitung der Conceptual Models

Hinsichtlich der bisher aufgestellten Anforderungen werden konzeptuelle Modelle für die verschiedenen Präsentationslogiken veranschaulicht.

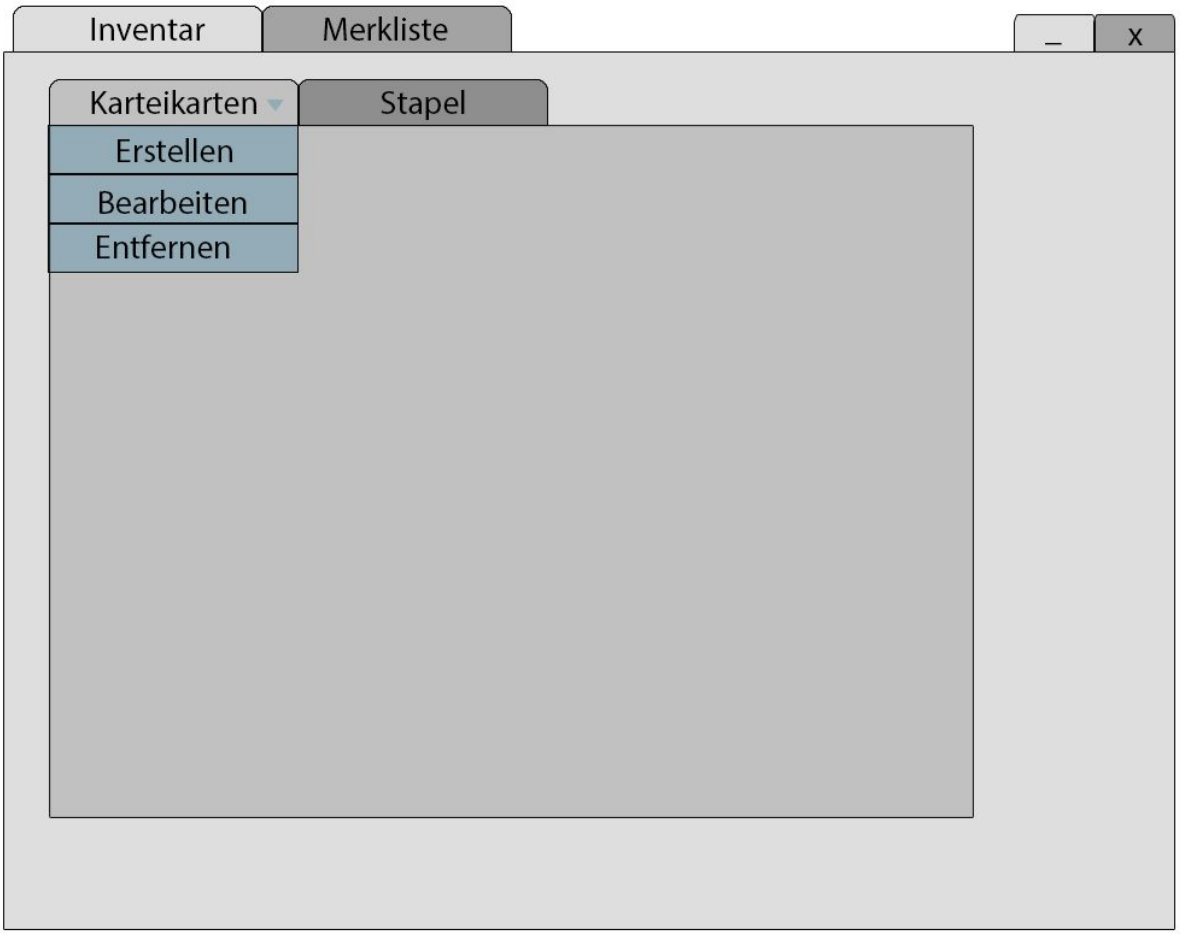


Abb. 10: Conceptual Model Desktopanwendung

Abb.11: CM1 Androidanwendung

Abb. 12: CM2 Androidanwendung

Durch die Gestaltung der Modelle ist aufgefallen, dass ein Zugriff auf die Karteikarten immer ermöglicht werden wollte, bevor die Lerneinheit startet. Zum Start der Lerneinheit soll jedoch ein Abbruch ermöglicht werden, der im Conceptual Model noch nicht erläutert wurde.

10.5 Fazit

Während der Bearbeitung des konzeptuellen Modells ist genauer auf Gestaltungselemente eingegangen, wodurch sich ein Richtwert für noch folgende Interface Gestaltungen ergeben hat. Da nun Ausarbeitungen aus dem Work Reengineering Prozess visuell modelliert werden konnten, haben sich Schwächen im System gezeigt: Eine Android Präsentationslogik, die alle Funktionalitäten der Anwendung erfüllt, ist bislang nicht festgelegt und wird im weiteren Verlauf wahrscheinlich noch erarbeitet werden müssen.

11. Screen Design Standards

Nach Mayhew werden die Screen Design Standards in sechs Unterkapitel für ein individuelles Design gegliedert:

- Control standards
- Procress windows standards
- Dialog Box contents standards
- Message box contents standards
- Input device interactions standards
- Feedback standards

Diese Standards müssen separat für die Desktopanwendung, sowie die Android Applikation festgelegt werden. Durch Googles hauseigenes Material Design wird einiges schon vorab für die Android Applikation eingegrenzt sein.[3] Jedoch lagen für die Desktopanwendung keine Vorlagen bereit, was dazu führt, dass die unten stehenden Screen Design Standards noch einmal festgelegt werden mussten.

11.1 SDS - Mobile Anwendung

Zur Orientierung des Designs wird grundlegend auf das Material Design von Google eingegangen, da dieses schon viele Vorgaben besitzt. Weil das Material Design eine Grundlage bietet, werden zusätzlich ausgewählte Produktfeatures, wie Farben, festgelegt. Unter Anderem ergibt das Eingehen auf Dialogboxen keinen Sinn, da diese im Material Design nicht fensterbasierend dargestellt werden. Stattdessen haben sich die Entwickler darauf fokussiert einige Standards für vereinzelte Screens festzulegen.

Control Standards
Um eine homogene Konsistenz im Bezug auf die Auswahlmöglichkeiten zu gewährleisten, beschreiben die Control Standards, die zu verwendeten Objekte dahingehend, dass ein Nutzer direkt erkennt, welche Funktionen gleichbleibende Objekte ausführen können.

Allgemeine Standards
Für die Farbe des Hintergrundes eines Textes der Screens soll eine Farbe des Standardsets (grau, weiß) aufgrund der besseren Lesbarkeit durch eine weiße, oder schwarze Schriftfarbe verwendet werden.

Tabelle 15: Mobile Menu Content

Menu Content	Control
Input des Users	Textfelder, Buttons
Navigation	Buttons
Auswahl Ja / Nein und An / Aus	Toggle Buttons
Objekte aus einer Auswahl	Spinners
Auswahl Datum und Uhrzeit	Pickers
Eingabe: Variables	Textfeld
Auswahl: Große Liste	Textfeld mit autom. Eingabevorschläge
Auswahl eines Objekts	Radio Buttons
Auswahl mehrerer Objekte	Check Buttons

Sowohl die Appbar als auch die Bottombar werden homogen in der Primärfarbe gehalten. Um eine klare Ablesbarkeit der Icons und Schriften zu gewährleisten, werden diese als weißer Negativtext und Negativicons dargestellt. Zudem ist der Text in der Appbar zentriert und die Navigationselemente, werden rechts oben platziert.
Mit der Berührung von Buttons und Elemente der Navigation, wird eine entsprechende Animation ausgeführt und ein Farbwechsel zur Sekundärfarbe wird vollzogen. Elemente die nicht mehr ausgewählt sind, nehmen ihre ursprüngliche weiße Farbe wieder an. Buttons werden immer zentriert und unter dem durchzuführenden Interaktion angeordnet.
Objekte die zusammen gehören, werden in vertikalen Gruppen geordnet, solange man sich im vertikalen und nicht im Landscapemodus befindet. Die Trennung der Objekte erfolgt mit Spacern, welche aus leichten grauen Linien bestehen.
Elemente können nur nach unten hin aus dem Screen rausgescrollt werden. Die obere Zeile schließt an die fixierte Appbar an.
Eingabefelder werden immer mit einem weißen Hintergrund angezeigt.

Message box content standards
Für die mobile Anwendung werden Notifications als Form von Benachrichtigungen, Toast als kurze Bestätigungen des Status der ausgeführten Interaktion und Alerts, welche einen Prozess unterbrechen, damit relevante Informationen eingegeben werden könne.
Toasts werden am unteren Bildschirmrand erscheinen und schieben alle Interaktionsobjekte für eine kurze Zeitdauer nach oben. Alerts hingegen nur, wenn eine Eingabe des Benutzers zwingend erforderlich sei.

Input device interaction standards
Die Eingabe durch den Benutzer wird auf dem Smartphone grundlegend mit dem Touchscreen erfasst. Als Eingabefläche dient die vorinstallierte Bildschirm- und Gesturetastatur.

Feedback standards
Google definiert hier schon einiges an Möglichkeiten des Feedbacks vor. Diese dienen diesem Projekt als Orientierung.

11.2 SDS - Desktopanwendung

Die Desktopanwendung wird standardgemäß mit Javascript für die Anwendungslogik, sowie HTML und CSS für das Design realisiert. Dies bedeutet, dass wir designtechnisch auch aufwendigere Screens erstellen könnten. Dies spielt in diesem Projekt jedoch eine untergeordnete Rolle, da der einzusetzende Workload hierfür nicht ausreicht. Viel wichtiger ist die Eingabe und Erhaltung sowie Verarbeitung der Daten.

Control standards
Um eine einheitliche Konsistenz in den Auswahlmöglichkeiten bieten zu können, müssen sogenannte Control standards eingeführt werden, die beschreiben, wie Objekte screenübergreifend aussehen.

Dialog box contents standards
Bei einer Desktopanwendung, welche in den meisten Fällen fensterbasierend ist, ergibt die Erstellung von Standards von Dialogboxen deutlich mehr Sinn. Dialogboxen sind unter Anderem temporäre Fenster, die die Auswahl von Optionen ermöglichen oder aber auch die aktuellen Werte für weitere Vorgänge festlegt:

- Zusammengehörige Felder werden in Gruppen angeordnet.
- Der Weißraum zwischen den Spacern soll bei Listen genutzt werden, um die einzelnen Objekte besser voneinander trennen zu können.
- Hintergrundfarben von Eingabefelder werden immer weiß gehalten, falls es optionale Felder sind, werden diese in einem leichten Grauton eingefärbt.

Message standards
Am unteren Bildschirmrand werden jeweilige Meldung angezeigt, die anzeigen, ob Karteikartenstapel erfolgreich gespeichert oder Lerneinheit gestartet wurden. Diese Meldungen lassen sich auch in den Einstellungen deaktivieren.

Input device interaction standards
Peripheriegeräte, wie Maus und Tastatur, sowie virtuelle Tastaturen werden für die Eingabe benötigt.

Feedback standards
Aktuelle Objekte werden mit einer helleren Farbe zur besseren Unterscheidung dargestellt. Buttons werden durch ein aufblinken in einer dunkleren Farbe als bedient markiert.
So ist immer ersichtlich, ob eine Interaktion übernommen wurde.

12. Detailed User Interface Design

Um den Usability Engineering Lifecycle erstmalig abzuschließen, wird im letzten Schritt ein Detailed User Interface Design vorgesehen. In diesem Artefakt werden alle bisher gewonnen Erkenntnisse umgesetzt und in Form von einer vollendeten Gestaltung der Nutzerschnittstelle präsentiert. Im Fokus liegt die Maximierung der Gebrauchstauglichkeit, indem Ergebnisse des Lifecycles angewandt und evaluiert werden.

Die folgenden Abschnitte befassen sich mit den Arbeitsschritten, die von Deborah Mayhew in einem Schritt-für-Schritt Vorgehen vorgeschlagen werden. Es wird zwischen mobiler Anwendung und Desktopanwendung unterschieden, da jeweils verschiedene Präsentationslogiken vorgesehen sind.

12.1 Mobile Anwendung

Zuerst sollen Navigationspfade zu Fenstern, Dialog und Message Boxes modelliert werden. Mayhew erwähnt hierbei, dass jeder einzelne Navigationsweg modelliert werden sollte. Aufgrund von Redundanz und der Skalierbarkeit des Vorgehensmodells auf kleinere Projekte, hat sich das Projektteam dagegen entschlossen.

Zu Beginn wurde sich an den Gestaltungs-Prototypen der konzeptuellen Modelle orientiert. Es ist aufgefallen, dass viele wichtige Bereiche, wie der Zugriff auf das eigene Nutzerkonto oder auch die Unterteilung der Suchvorgänge, nicht dargestellt wurden. Das sehr simpel gestaltete Design hat sich durch die Gestaltungsprinzipien “Only show what I need when I need it” und “Material Design maintains the same UI across platforms”, aus Kapitel 8, definiert. Die festgelegten Screen Design Standards haben hierfür nötige Details, wie die Button-Gestaltung oder Farbgestaltung, unterstützt.

Navigationswege
Es wurde sich dazu entschieden, das Menü-Icon in der rechten oberen Bildecke abzubilden (siehe Abb. 13), anstelle von der linken. Die Entscheidung wurde getroffen, da die meisten Smartphone-Nutzer Rechtshänder sind und somit das Icon besser mit dem Daumen erreichen können.
Um die Suche von Karteikarten und Karteikartenstapeln effektiver zu gestalten, wird über das Seitenmenü (siehe Abb. 14) ermöglicht, sowohl über Semester, als auch einer klassischen Suche, Objekte zu finden. Semester haben ein Pull-Down Menü und können die jeweiligen Fächer anzeigen, unter denen Karteikartenstapel zu finden sind.
Mit einem Zurück-Icon an der oberen linken Ecke lässt sich die übergeordnete Seite aufrufen.

Ursprünglich war geplant diese Funktion rauszunehmen, da Smartphones bereits über die integrierten Touch-Tasten die gleiche Funktionalität bieten. Jedoch wurde sich dafür entschieden dem Nutzer zu verdeutlichen, dass dieser sich auf einer Unterseite befindet und ebenfalls nach Betätigen des Zurück-Icons auf die Startseite zurückfinden kann.

Das Starten einer Lerneinheit geschieht erst nach Bestätigung der Auswahl einer Check-Box, die auf einen Karteikartenstapel verweist. Somit wird ein Zwischenschritt eingeleitet, der vermeiden soll, dass Nutzer aus Versehen einen falschen Karteikartenstapel wählen.

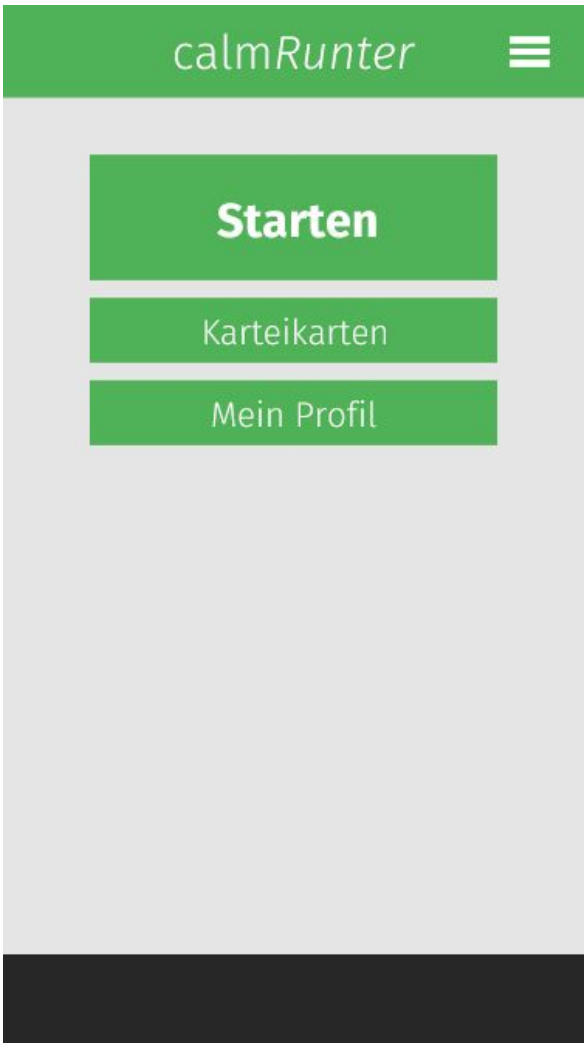


Abb. 13: Startscreen calmRunter

Abb. 14: Seitenmenü calmRunter

Lernprozess

Während das Lernsystem als primäre Funktion weiterhin bestehen bleibt, wurde eine Bottombar hinzugefügt, die mehrere Funktionen, wie das Suchen von Karteikarten und Karteikartenstapel oder Beginnen einer Lerneinheit (siehe Abb. 15), beinhaltet. Bei der Gestaltung des Lernsystems konnten Ergebnisse der Contextual Task Analysis visuell dargestellt werden, was den Gestaltungsprozess erleichtert hat.

Der Nutzer kann einen gespeicherten Karteikartenstapel auswählen und bekommt über der Bottombar angezeigt wie viele Minuten die Lerneinheit ungefähr lang sein wird (siehe Abb. 16). Auf dem darauf folgenden Screen wird der Lernprozess abgebildet.

Durch die Auswahl von Präferenzen kann ein Nutzer den Lernzyklus beeinflussen. Es wurde sich dazu entschieden Präferenzen durch das Seitenmenü aufrufen zu können und nicht im eigenen Profil festzulegen, damit diese Einstellungen zu jeder Zeit änderbar sind.

Zu Beginn war es den Entwicklern unklar, ob die Google Maps Karte als Vollbild angezeigt werden sollte. Da sich bei dieser Variante aber Probleme mit dem Lernen von Karteikarten ergeben, wurde sich für eine andere Variante entschieden, die sowohl die Karte, als auch Karteikarten auf einem Screen vereint.

Abb. 15: Lerneinheit calmRunter

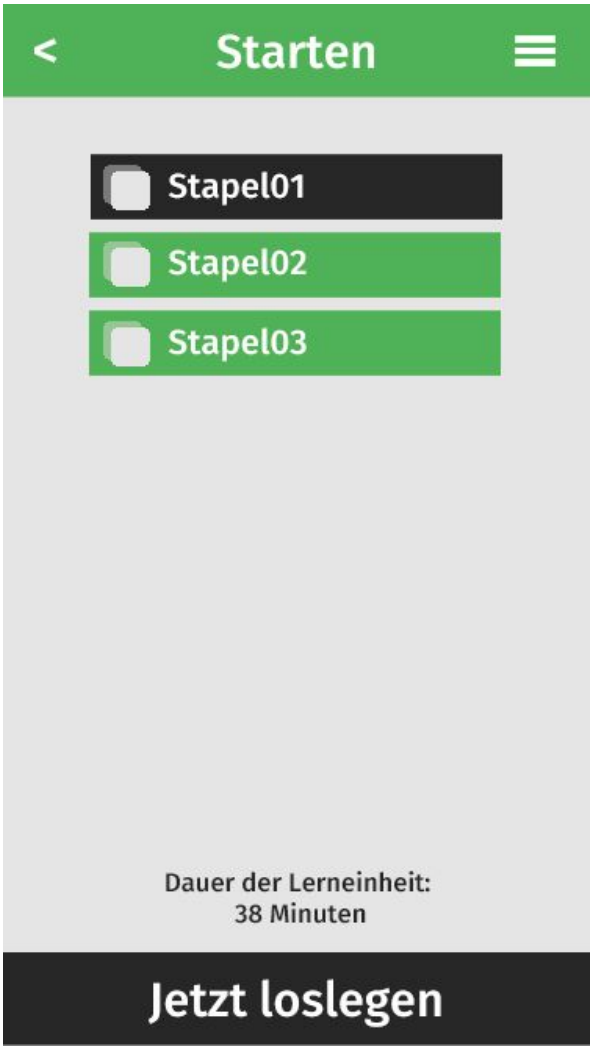


Abb. 16: Starten calmRunter

Karteikartensystem

Weiterhin ist die Gestaltung der Karteikartenverwaltung von wichtiger Bedeutung. Der Nutzer soll ohne große Umstände einen Überblick von seinen gespeicherten Karteikarten bekommen und diese schnell verwalten können, um den Usability Goals gerecht zu werden. Durch den Work Reengineering Prozess und das darin verbesserte Hierarchical Task Analysis Modell ließ sich die Gestaltung Schritt für Schritt von den Ebenen der Karteikartenverwaltung ableiten.

Schwierig fiel jedoch die Darstellung des Speichervorgangs von fremden Karteikarten oder Stapeln. Zu Beginn stand ein Toggle-Button und ein Speicher-Button zur Auswahl. Letztendlich wurde sich für den Speicher-Button entschieden, um einen größeren Bereich für die Touch-Gesten zu haben.
Um veröffentlichte Objekte von anderen Nutzern zu suchen, dient eine Suchleiste in der Bottombar des Screens (siehe Abb. 17). Nutzer können durch Tags, Namen der Objekte und Nutzernamen die Sucherergebnisse filtern (siehe Abb. 18).
Zur eigenen Erstellung von Karteikarten sind Textfelder für Namen, Vorder- und Rückseite vorgesehen (siehe Abb. 19, 20) , die mit einem “Speichern” Button in der Bottombar abgespeichert werden können. Das Erstellen von Karteikartenstapeln zeigt alle eigenen Karteikarten an, die erstellt oder gespeichert wurden. Der Nutzer kann durch antippen einzelne Karteikarten markieren und in der Bottombar auf die Schaltfläche “Erstellen” tippen. Um eine Auswahl zu markieren, findet ein Farbwechsel der Primärfarbe bei Karteikarten statt, wie in den Screen Design Standards beschrieben.

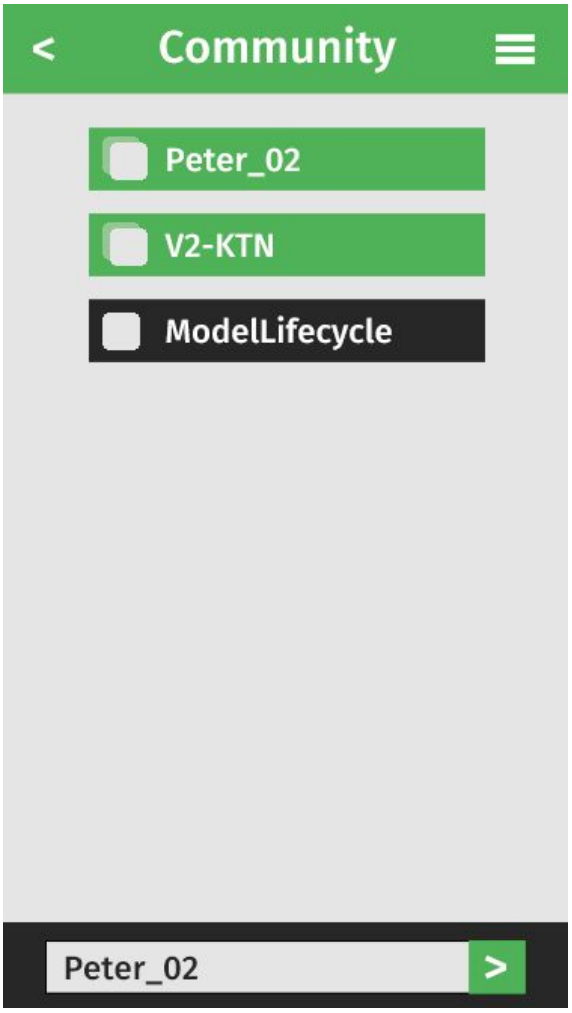
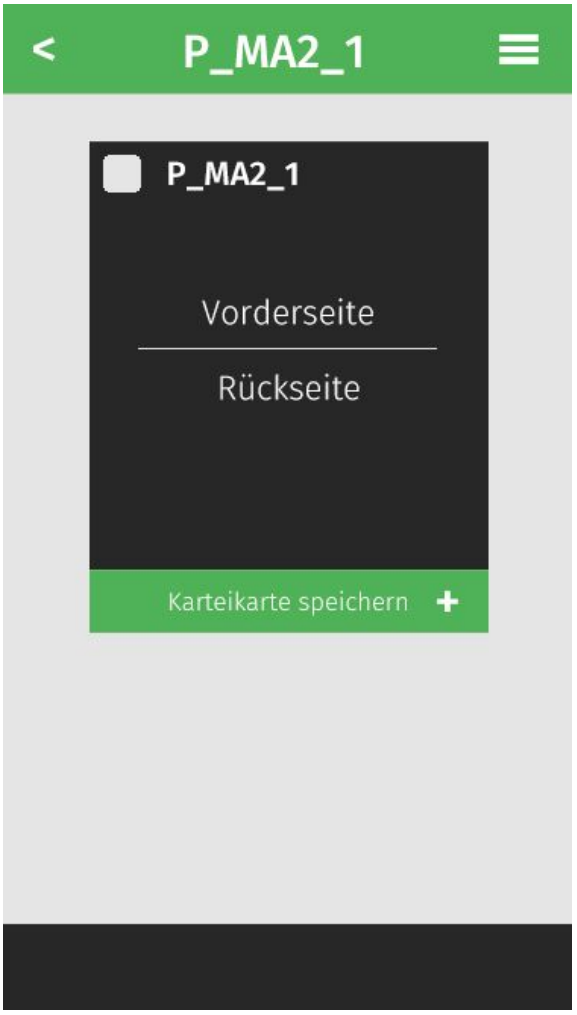


Abb. 17: Community calmRunter

Abb. 18: Karteikartenstapel calmRunter



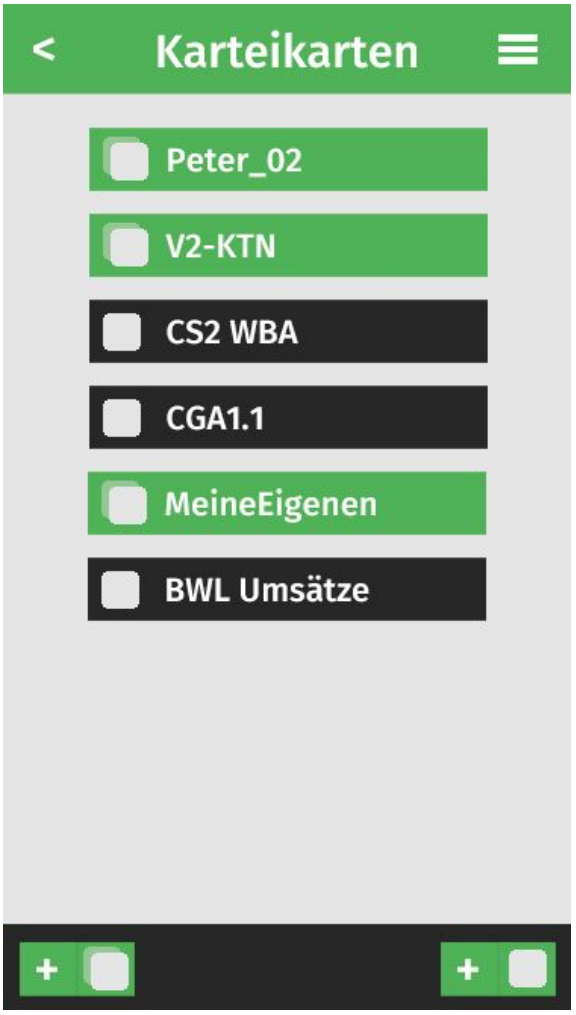


Abb 19: Karteikarte calmRunter

Abb. 20: Karteikarten calmRunter

Das eigene Profil benötigt keine Untergliederung und ist deshalb nicht aufgeführt.

12.2 Desktopanwendung

In diesem Kapitel wird die Gestaltung der Desktopanwendung dokumentiert.
Vor dem Beginn wird das 2. aufgezählte Gestaltungsprinzip, aus Kapitel 8, beachtet, um eine einheitliche Gestaltung auf verschiedenen Endgeräten vorzunehmen. Durch diesen Schritt werden Elemente benutzt, die bereits in der mobilen Anwendung vorhanden sind. Um Redundanz zu vermeiden werden folglich nur die Screens modelliert, welche die Funktionalität der Desktopanwendung verdeutlichen.

Die Gestaltung wurde stark von dem konzeptuellen Modell beeinflusst, wodurch ein Hierarchiesystem veranschaulicht wurde. Da die Desktopanwendung eine reine Kontrollmöglichkeit über Karteikartenstapel für den Nutzer bieten soll, wurde die Spazier-Komponente ausgelassen.
Gleich zu Beginn wurden die primären Reiter für die Fenster-Leiste festgelegt. Im Bereich “Mein Profil” sollen nur die wesentlichen Informationen gegeben werden über das Nutzerkonto, ähnlich wie bei der mobilen Anwendung.

Eine Modellierung für diesen Bereich ist nicht vorgesehen.
Für die Karteikartenverwaltung ist eine simple, hierarchische Struktur vorgesehen (siehe Abb. 21).
Das Team war erstmals unentschlossen einen weiteren Reiter für Karteikarten hinzuzufügen. Nachdem jedoch die Karteikartenstapel modelliert wurden, ist ein eigener Reiter für Karteikarten nicht nötig. Karteikarten werden bereits unter Karteikartenstapeln aufgelistet und können bearbeitet und veröffentlicht werden.
Während der Modellierung ist aufgefallen, dass Tags, Semester und das Fach gesetzt werden müssen. Semester und Fach wurden in den Ausarbeitungen des Usability Engineering Lifecycles nicht berücksichtigt, sind aber nötig für die Suche.
Ausgewählte Reiter werden in der Primärfarbe dargestellt, während nicht ausgewählte Reiter eine andere Farbe aufweisen und mit einem Spacer unterteilt werden.
Neue Karteikartenstapel und Karteikarten können mit dem unterstem Reiter erstellt werden. Hierzu bildet sich ein neues Element, welches benannt werden muss.
Die Interaktion des Nutzers findet ausschließlich durch Texteingabe und Mausklicks statt.

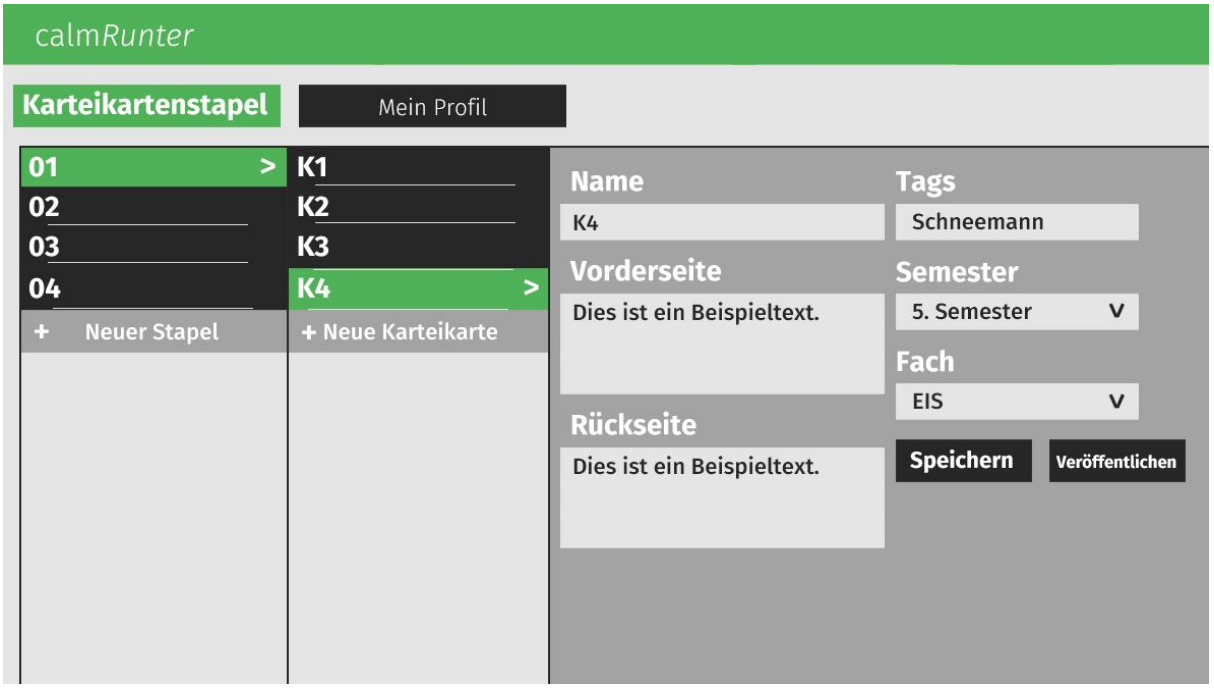


Abb. 21: Desktopanwendung calmRunter

Fazit
Die Hauptfunktionen der Anwendungen sind dadurch abgedeckt und können im weiteren Projektverlauf evaluiert und korrigiert werden.

13. Systemdokumentation

Die Systemdokumentation befasst sich mit der Iteration des WBA-Teils in Bezug auf den erarbeiten Erkenntnissen der Mensch-Computer Interaktion und des Rapid Prototypes. In den folgenden Abschnitten werden bereits fertiggestellte Modelle aus der Konzeption überarbeitet und abgewägt, sowie Vertiefungen zu projektspezifischen Themen beschrieben.

Im Hinblick auf das Konzept hat sich die ausgewählte Domäne kaum verändert. Mit der Betrachtung von weiteren sekundären Stakeholdern, als Ergebnis des Vorgehensmodells, hat sich ein abgeändertes Kommunikationsmodell gebildet, welches nun weitere Akteure beinhaltet. Im folgenden Abschnitt wird die veränderte Interaktion mit dem System dargestellt.

13.1 Kommunikationsmodelle

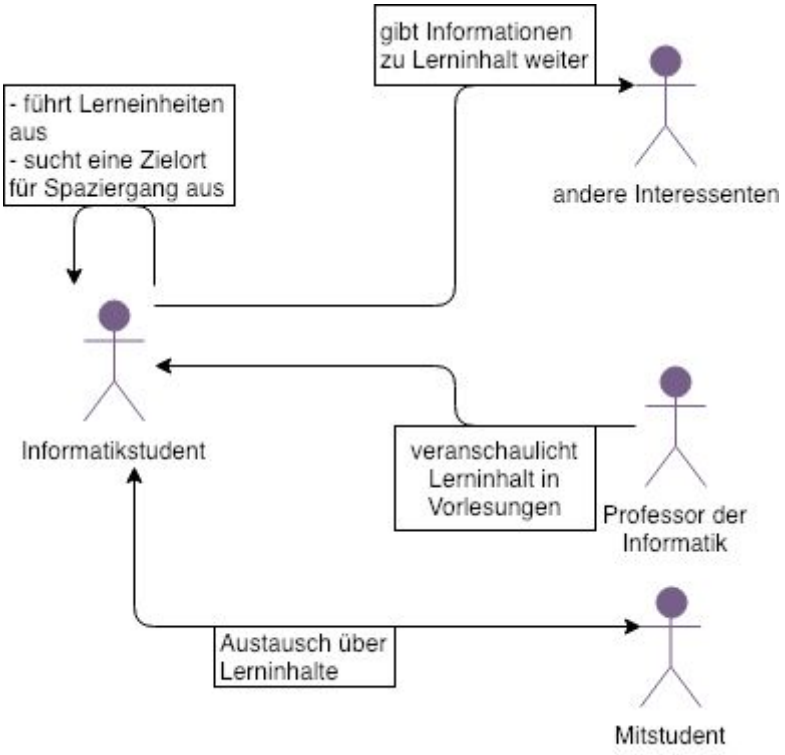


Abb. 22: Deskriptives Kommunikationsmodell

Das deskriptive Kommunikationsmodell (siehe Abb. 22) wurde bereits im Konzept fehlinterpretiert dargestellt. Anders als im Konzept, ist dieses Modell nun technologiefrei veranschaulicht. Der Unterschied zum präskriptiven verdeutlicht hiermit die Funktionalität des Anwendungssystems und wie dieses die Akteure unterstützt.

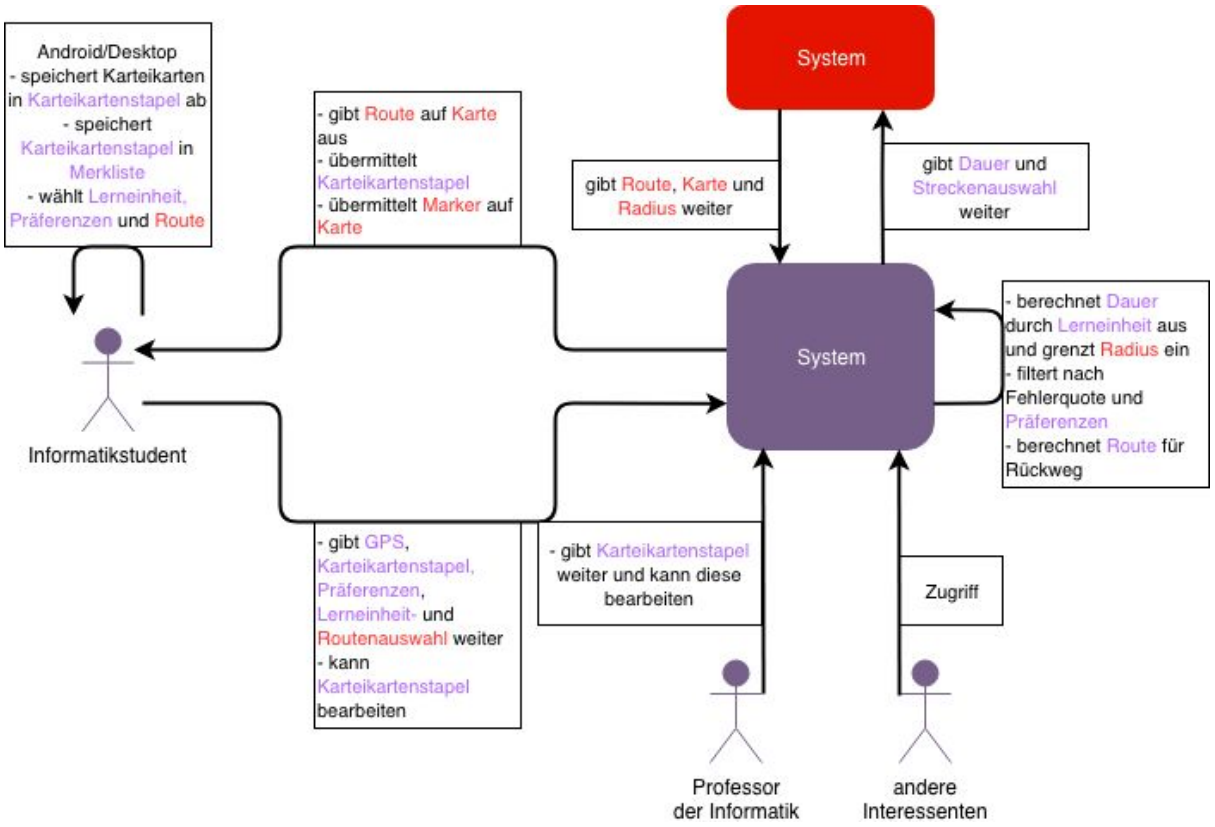


Abb. 23: Präskriptives Kommunikationsmodell

Für das präskriptive Kommunikationsmodell (siehe Abb. 23) sind weitere Funktionen des Anwendungssystems veranschaulicht. Angefangen beim Informatikstudent hat dieser eine Merkliste, auf der Karteikartenstapel gespeichert werden, unabhängig ob sie die eigenen oder fremden Karteikartenstapel sind. Professoren der Informatik und Informatikstudenten können Karteikartenstapel auf dem System ebenfalls bearbeiten.
Das System bietet in Bezug auf die Anwendungslogik eine Filterung an, die nicht nur eine Priorisierung anhand der Fehlerquote des Lernenden festlegt, sondern auch die eigenen Präferenzen des Studenten berücksichtigen soll.
Wurde die Lerneinheit vom Studenten gestartet und der ausgewählte Marker erreicht, soll die Karte automatisch einen Rückweg anzeigen können, der im zeitlichen Rahmen der Lerndauer liegt und, anders als der Hinweg, zum Ursprung führt.
Ein wesentlicher Unterschied ist der Zugriff durch andere Nutzer. Die in den User Profiles erwähnten Professoren der Informatik und weitere Interessenten können mit dem System interagieren. Andere Interessenten wurden hierbei jedoch nicht detailliert dargestellt, da diese nur sekundäre Stakeholder sind und ähnliche Interaktionen, wie ein Informatikstudent, aufweisen.
Ebenfalls werden zuvor angesprochene Zeitmessungen, wie die Pomodoro-Technik, vorerst rausgelassen und nicht visuell dargestellt, da die Entwickler andere Anwendungslogik als wichtiger eingestuft hat.

Den Entwicklern war es sehr wichtig die Ergebnisse des Vorgehensmodells und die Kritik aus Besprechungen mit Projektbetreuern bei der Überarbeitung zu berücksichtigen. Schwierig war hierbei besonders die Auswahl der Anwendungslogik, da im Vorfeld auf einen

eventuellen Mangel für den Prototypen hingewiesen wurde. Es mussten also Logiken gefunden werden, die die Gebrauchstauglichkeit unterstützen und die Zeitplanung berücksichtigen. In den folgenden Abschnitten wird genauerer Bezug zu den einzelnen Komponenten des zukünftigen Prototyps genommen.

13.2 Architekturmodell

Durch eine Iteration des vorherigen Architekturmodells wurde dieses abgeändert (siehe Abb. 24). Bereits zuvor wurden einige Komponenten des Architekturmodells im Konzept fehlinterpretiert. Bei der Vorversion des Architekturmodells wurde nicht verdeutlicht, dass eine synchrone Kommunikation geplant ist.
Hierbei bestand die Schwierigkeit für das Team darin, den präskriptiven Zustand einzuschätzen, ohne sich mit der Abfolge des Datenaustauschs auseinandergesetzt zu haben. Die aktuelle Ausarbeitung der Gestaltung setzt eine Veröffentlichung der Karteikartenstapel voraus, die synchron realisiert werden soll. In den folgenden Abschnitten wird ein genauerer Bezug zu den Änderungen des Architekturmodells genommen.

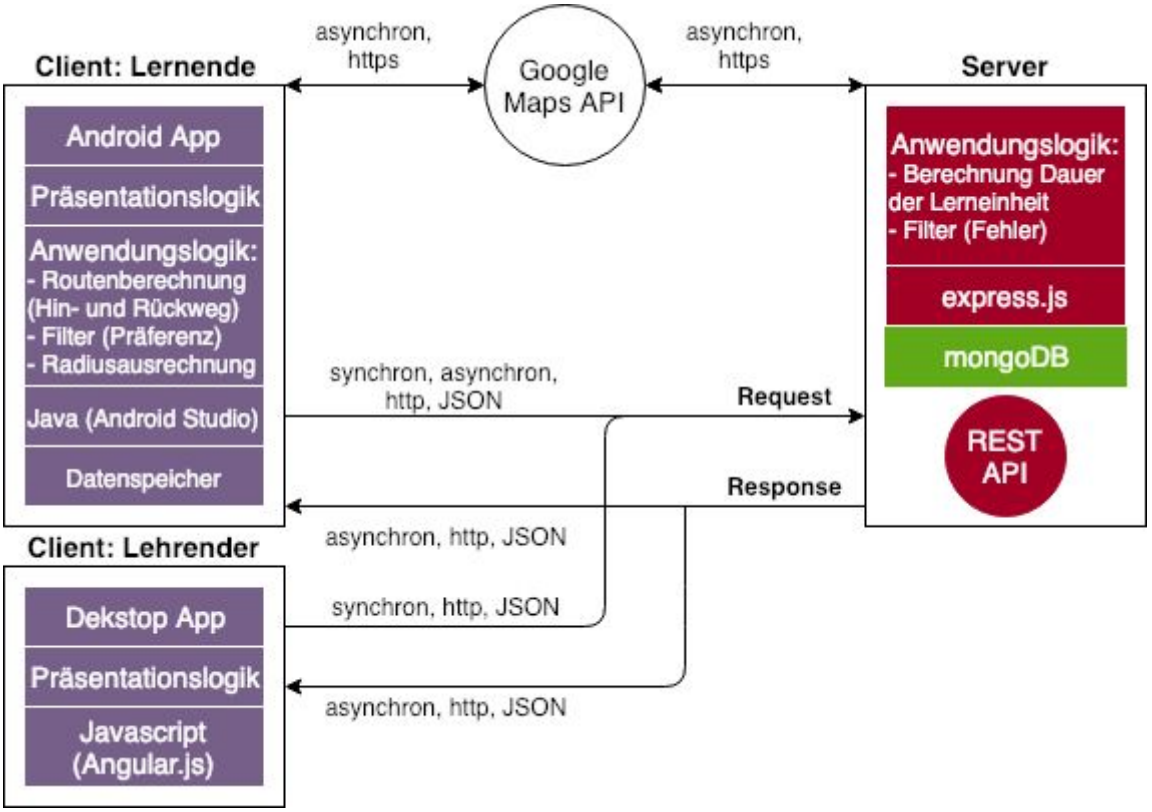


Abb. 24: Iteration der Systemarchitektur

13.2.1 Iteration der Komponenten

Die durch Iteration überarbeiteten und neuen Komponenten der Systemarchitektur werden in der obenstehenden Abbildung veranschaulicht.

Clients
Im Gegensatz zu der ersten Systemarchitektur wurde nun der Client aufgeteilt in eine Lernenden-Komponente und eine Lehrenden-Komponente. Die unterschiedlichen Clients differenzieren sich in der Art der Anwendung. Während Lernende von einer mobilen Android App profitieren, ziehen Lehrende einen Vorteil durch eine übersichtliche Desktopanwendung.
Der Javascript Teil des Clients für Lehrende soll mit Angular.js realisiert werden, wohingegen für die Entwicklung des Clients für Lernende Java vorgesehen ist. Die Umsetzung erfolgt in der frei verfügbaren Entwicklungsumgebung Android Studio, da ein Teammitglied hiermit bereits Erfahrung gesammelt hat.
Da beide Clients auf verschiedenen Endgeräten verfügbar sein werden, sind unterschiedliche Präsentationslogiken notwendig.
Ebenfalls unterscheidet sich die Anwendungslogik, da der Lehrende nur Daten zur Datenbank des Servers schickt, die von Lehrenden verwendet werden können. Somit besteht auch keine Kommunikation zwischen Google Maps API und Lehrenden.
Damit Lernende unterwegs keine mobilen Daten verbrauchen müssen um das Anwendungssystem zu nutzen, ist eine lokale Speicherung vorgesehen, die Karteikartenstapel und Karten offline verfügbar machen kann.

Server
Der Server beinhaltet ebenfalls Anwendungslogik, die sich von der des Clients unterscheidet und steht in Verbindung mit beiden Clients, um Daten aus der Datenbank zu übertragen. Die REST API ist in dieser Systemarchitektur anders positioniert, da dieser Dienst sowohl eine Anbindungsmöglichkeit für den Client, als auch der Google Maps API, darstellt.

13.2.2 Kommunikation

Server
Für die Kommunikation des Servers wurde sich für eine REST-Architektur entschieden, um mit den generischen HTTP-Verben des nodeJS Moduls Express zu kommunizieren. Express besitzt eine ressourcensparende und gleichzeitig umfassende Bibliothek. Dem Projekt werden somit viele Möglichkeiten gegeben durch weitere Module, falls nötig, den Entwicklungsprozess effektiver zu gestalten.
Der Server kommuniziert mit einer Google Maps API, um einen Radius für jeden, in der Datenbank gespeicherten, Karteikartenstapel einzugrenzen und abzuspeichern. Damit müssen keine neuen Berechnungen für die Auswahl eines Karteikartenstapels durchgeführt werden. Dieser Vorgang passiert asynchron, damit der Server weitere eingehende Anfragen vom Client abarbeiten kann und damit zeitlich und referentiell entkoppelt ist. Der

eingegrenzte Radius wird dann asynchron über Google Maps API an den Client weitergeleitet, damit mögliche Alternativen für einen Spaziergang sichtbar sind.
Für die Kommunikation mit den Clients werden die Daten der Karteikartenstapel aus der Datenbank entnommen und als Response an die Clients verschickt. Hier wurde sich für eine asynchrone Kommunikation entschieden, damit die Übertragung von öffentlichen Daten den Server nicht vor weiteren Interaktionen stoppt.

Client
Der Server erhält dementsprechend GET oder PUT Requests vom Client und verändert die Datensätze. Aus Gründen der Effizienz werden Ressources nicht komplett ausgegeben oder manipuliert. Weiterhin führt Paginierung, also die Ausgabe von Teilen einer Ressource, für eine effizienteres System.
Ein Request kann sowohl synchron, als auch asynchron geschehen. Während GET-Methoden synchron auf einen Datensatz warten, ist für den Client via PUT-Methode keine Rückmeldung von Nöten. Werden beispielsweise alle öffentlichen Karteikartenstapel gesucht, warten beide Endpunkte der Kommunikation auf eine erfolgreiche Datenübertragung um die gewünschten Informationen sofort zur Verfügung zu erstellen. Bei einer Änderung von eigenen Karteikartenstapeln, die öffentlich verfügbar sind, werden PUT-Methoden nach einem Warteschlangen Prinzip synchron vom Server abgearbeitet.
Der Client für Lernende kommuniziert ebenfalls mit der Google Maps API. Asynchron werden hierbei alle nötigen Daten von der API empfangen, um die Spazierroute richtig darstellen zu können.
Bezüglich des Datenschutzes werden abgesehen von den GPS-Position keine persönlichen Daten des Nutzers gespeichert. Die Ermittlung der GPS-Position erfolgt nur dann, wenn Zugriff für die mobile Anwendung vom Nutzer genehmigt wird.

13.3 Datenbank

Um Daten persistent speichern zu können, wurde sich für MongoDB entschieden. Diese Entscheidung fiel allerdings nicht leicht, da zu Beginn der Dokumentation zwischen MongoDB, MySQL und Redis entschieden werden musste.

Redis

Redis zeichnet sich als “Key Value Store” aus und ermöglicht es Daten mit einem Schlüssel zu versehen und dadurch Datenstrukturen effektiver abzufragen und organisieren zu können. Dies könnte wichtig für eine spezifische Abfrage von Karteikartenstapeln im System sein. Weiterhin ist Redis ressourcenschonend und eine der am meisten genutzte NoSQL Datenbank im World Wide Web (Quelle). Sollten im Projektverlauf Probleme mit Redis auftauchen, wird sehr wahrscheinlich eine Lösung im Internet zu finden sein.

MySQL

Mit den inbegriffenen JOIN-Operatoren wird eine zufriedenstellende Datenbankpflege von MySQL vorausgesetzt. Weiterhin verfügt MySQL über ein passwortgeschütztes Sicherheitssystem, welches eine sichere Umgebung für sensible Daten bietet.
Das Projektteam hat bereits Erfahrung im Verlauf des Studiums mit MySQL sammeln können und hat diese Datenbank abgewiesen, wegen schwieriger Skalierbarkeit und nicht ausreichender Hilfestellung der Community.

MongoDB

Mit MongoDB würde keine eindeutige Identifizierung von Daten vorausgesetzt sein, was mit einem selbstdefinierten Schlüssel der Entwickler behoben werden kann. Zwar bietet MongoDB keine vergleichbar schnelle Antwortzeit, benötigt aber einen nicht so hohen Lernaufwand, wie Redis, dank einer traditionellen und simpel strukturierten Datenbank. Die Verwendung von JSON Objekten findet bei MongoDB ebenfalls Unterstützung und kann somit zeitsparend integriert werden.

Abwägung

Insgesamt bietet MongoDB ausreichende Funktionen für das Anwendungssystem. Zwar ist ein Schlüssel für die Datenbank hilfreich, aber nicht unbedingt nötig für die Pflege der Datenbanken. Wie bereits im Vorgehensmodell beschrieben, sollen Filterungen bei der Suche eines Karteikartenstapels ermöglicht werden, die sich nicht nach einem Primärschlüssel richten, sondern nach Tags oder Suchwörter. Wichtig ist für den weiteren Verlauf des Projekts ebenfalls die Verwendung von JSON Objekten, da beide Teammitglieder hiermit bereits Erfahrung gesammelt haben. Redis unterstützt jedoch nur die Verwendung von Strings und Hashes, um die Datenpflege zu optimieren. Hierzu müssten sich die Teammitglieder intensiv beschäftigen, wodurch mehr Zeit für eine Lösung investiert wird, die durch MongoDB umgangen wird.

Middleware

Um eine Interaktion mit der Datenbank zu ermöglichen, wird das Node Modul mongoose.js verwendet.

13.4 Datenstrukturen

Für das Anwendungssystem sind Karteikarten und Karteikartenstapel in der Datenbank als JSON Objekte hinterlegt und haben für das Projekt hohe Wichtigkeit. In diesem Abschnitt wird genauer auf die Struktur dieser Objekte eingegangen.

Karteikarte

Karteikarten werden von Nutzern angelegt und haben folgende Struktur:

```
karteikarte
{
  id: Number,
  ersteller: String,
  vorderseite: String,
  rueckseite: String,
  laenge: Number
}
```

Abb.25: Datenstruktur Karteikarte

Das Attribut “id” wird automatisch beim Erstellen der Karteikarte inkrementiert und dient, im Zusammenspiel mit dem Namen des Erstellers, zur eindeutigen Identifikation einer Karteikarte. Vorder- und Rückseite beinhalten die vom Nutzer eingetragenen Informationen der Karteikarte. Wichtig ist hierbei, dass die Rückseite die Antwort auf die Vorderseite gibt. “laenge” gibt die Zeit ein, die für eine Karteikarte in Anspruch genommen wird.

Karteikartenstapel

Ein Karteikartenstapel besteht aus einer oder mehreren Karteikarten und kann sowohl vom Lernenden, als auch Lehrenden erstellt und veröffentlicht werden. Es wird folgende Struktur verwendet:

```
karteikartenstapel
{
  name: String,
  ersteller: String,
  anzahlKarten: Number,
  lerndauer: Number,
  karteikarte:{
                id: Number,
                ersteller: String,
                vorderseite: String,
                rueckseite: String,
                laenge: Number
              },
  tag: String,
  kategorie:{
              studienfach: String,
              modul: String
            }
}
```

Abb. 26: Karteikartenstapel Datenstruktur

Da Karteikartenstapel gesucht werden können und das entweder durch Namen oder tags, wurden diese Attribute hinzugefügt. Der Ersteller des Karteikartenstapels ist ebenfalls wichtig, um Lernende von Lehrenden zu unterscheiden und bestimmten Nutzernamen einen anderen Rang zu geben. “anzahlkarten” beschreibt die Anzahl der gespeicherten Karteikarten, welche im Karteikartenstapel enthalten sind. Die Lerndauer ergibt sich aus dem Attribut “laenge” der Karteikarten und ist nötig um den Radius der Markierungen zu berechnen. Um in bestimmten Modulen von Studienfächern Karteikartenstapel finden zu können, wurde das “kategorie” Attribut hinzugefügt.

Nutzer

Um Karteikarten und Karteikartenstapel zuordnen zu können, sind Nutzerdaten nötig:

```
nutzer
{
  nutzername: String,
  karteikartenstapel:
                      {
                       name: String,
                       ersteller: String
                      },
  koordinaten:{
              langitude: Number,
              longitude: Number
              }
}
```

Abb. 27: Nutzer Datenstruktur

Für den Nutzer ist der abgespeicherte Nutzername nötig, um sich authentifizieren zu können. Die gespeicherten Karteikartenstapel sind ebenfalls im Nutzer-Objekt gespeichert, um erkennen zu können, ob der Nutzer nicht nur eigene, sondern auch andere Karteikartenstapel gespeichert hat. Für die Routenberechnung sind die GPS-Koordinaten des Nutzers wichtig, um nicht nur die eigene Position ermitteln zu können, sondern auch den Fortschritt des Spaziergangs betrachten zu können.

Abwägung

Um Datenstrukturen in Relation zueinander darzustellen, hat sich die Möglichkeit geboten ein Entitiy-Relationship-Model anzufertigen. Da jedoch MongoDB zur Datenspeicherung verwendet wird, ist kein einheitliches Relationsschema vorgegeben. Somit erfolgt die Relation bereits in den JSON-Strukturen.

13.5 Anwendungslogik

Die Anwendungslogik wird über zwei Systemkomponenten verteilt: den Server und Lernenden Client. Während der Überarbeitung war es wichtig zwischen Funktionen zu unterscheiden, die mobil auf der Android Applikation und im Hintergrund auf dem Server ausgeführt werden. In den folgenden Abschnitten wird die Verteiltheit der Anwendungslogik genauer erläutert.

Client: Lernender

Mit der im Modell gezeigten Anwendungslogik des Clients werden Berechnungen und Filterungen auf Basis einer Android Anwendung angesprochen.
Der Client kommuniziert mit der Google Maps API und übermittelt seine GPS-Position um eine Routenberechnung mit der Dauer des Karteikartenstapels zu ermöglichen. Die Formel hierzu wurde bereits im Konzept veranschaulicht und erneut angepasst. Die Formeln werden als Pseudocode dargestellt, da zu diesem Zeitpunkt keine weiteren Schritte für die Implementierung unternommen wurden.

Routenberechnung:
Streckenlänge (m) = Dauer der Lerneinheit (min) x durchschnittliche Laufdistanz (m) pro Sekunde * 60/2

Dauer der Lerneinheit:
Die Dauer der Lerneinheit ergibt sich aus dem “laenge” Attribut der Karteikarten und wird mit folgender Formel dargestellt:

```
wiederhole
        Dauer der Lerneinheit (min) += Zeitdauer der Karteikarte
für jede Karteikarte
solange bis der Karteikartenstapel durchgelaufen ist
```

Rückwegberechnung:
Neu in der Anwendungslogik des Clients ist die Berechnung des Rückwegs. Hierbei ist zu beachten, dass ein Rundweg entstehen soll, also eine Route, die auf anderem Weg zurück zum ursprünglichen Standort kommt. Zuerst werden hierfür die nötigen Bedingungen aufgestellt:

- Suche Strecke von aktuellen Standpunkt, die nicht dem Hinweg entspricht
- Strecke muss zum ursprünglichen Standpunkt des Nutzers führen
- Die Streckenlänge muss in etwa der Streckenlänge des Hinwegs entsprechen

Aus den Bedingungen kann nun die Formel als Pseudocode dargestellt werden:
sucheRouteFürRückweg mit Ziel des Hinwegs als aktueller Standort und Ursprung als Zielpunkt

```
speichereErgebnisseInArray
wiederhole für jedes Element in Array
        wenn rückwegLänge ist gleich hinwegLänge ±5 Minuten
        dann
                wenn Rückweg nicht gleich Hinweg
                        zeigeRückweg
```

Filterung nach Präferenzen:
Weiterhin ist eine Filterungen nach den Präferenzen des Nutzers vorgesehen. Der Nutzer soll auf einem Screen auswählen können welche Fächer er für die nächste Lerneinheit bevorzugt. Sind Karteikartenstapel gemischt, werden die bevorzugten Fächer mit einer höheren Priorität dem Nutzer gezeigt und wiederholt.

- Der Nutzer wählt von von einer Skala 1-5 aus, wie wichtig ein Fach für ihn ist
- Die Karteikartenstapel werden in der Reihenfolge nach der Präferenz geordnet (je höher, umso häufiger)

Berechnung:

```
für jedes Element im Karteikartenstapel
wenn Präferenz niedriger ist als präferenzNächstesElement
        Speicher ist diesesElement
        diesesElement ist nächstesElement
        nächstesElement ist Variable
```

Wiederholung nach Präferenz Berechnung:

```
wenn Karteikarte korrekt beantwortet worden ist
        dann karteikartePräferenz - 1
        wenn karteikartePräferenz über 2
                dann wiederhole für jedes element des Karteikartenstapels
                        wenn karteikartePräferenz niedriger ist als nächstes Element
                                        dann Speicher ist diesesElement
                                        diesesElement ist nächstesElement
                                        nächstesElement ist Variable
                                        wiederhole_ende
                        sonst nächstes Element prüfen
        sonst karteikarte an das Ende des Karteikartenstapels setzen
sonst Karteikarte ist falsch beantwortet worden
        wenn karteikartePräferenz größer als 4.5
                dann karteikartePräferenz ist 5
                karteikarte an drittnächster Stelle im Stapel
        sonst
                karteikartePräferenz + 0,5
                wenn karteikartePräferenz über 2
                        dann wiederhole für jedes Element des Karteikartenstapels
                                wenn karteikartePräferenz niedriger ist als nächstes Element
```

```
                    dann Speicher ist diesesElement
                    diesesElement ist nächstesElement
                    nächstesElement ist Variable
                    wiederhole_ende
                sonst nächstes Element prüfen
        sonst Karteikarte an das Ende des Karteikartenstapels setzen
```

Die Suchmaske des Radius wurde bereits im Konzept erläutert und wird höchstens nur noch auf Fehler getestet für den Prototyp.

Server

Die Anwendungslogik des Servers bezieht sich auf Berechnungen, die im Hintergrund ablaufen. Unter anderem werden öffentliche Karteikartenstapel mit einer Dauer versehen, aber auch Karteikarten während des Lernens mit Fehlerquoten berücksichtigt.

Um die Dauer einer Lerneinheit zu bestimmen und damit die Route eines Spaziergangs, müssen die Zeitdauer-Werte der Karteikarten zusammengezählt werden. Wie oben bei der Dauer einer Lerneinheit gezeigt, wird eine ähnliche Berechnung auch serverseitig benutzt.

Sollten dem Nutzer das Verständnis bestimmter Karteikarten während dem Lernprozess schwerfallen, werden diese Karteikarten öfter wiederholt. Clientseitig wird diese Funktion bereits in der “Wiederholung nach Präferenz” Berechnung durch Pseudocode thematisiert. Die Präferenz steht hierbei für die Wichtigkeit der Wiederholung und steigt, wenn eine Karteikarte falsch beantwortet wird.

Abwägungen

Zu Beginn des Konzepts waren die Präferenzen des Nutzers als Anwendungslogik für den Server eingeplant. Da sich jedoch herausstellte, dass das primär genutzte Endgerät ein Mobilgerät ist und die Anwendung auch offline verfügbar sein sollte, wurde die Anwendungslogik dem Client zugeteilt.
Weiterhin ist auch die Funktion der Pauseneinteilung, die im Konzept erwähnte Pomodoro-Technik, entfallen. Zwar könnte diese Funktion für mehr Zufriedenheit des Nutzers sorgen, jedoch ist das optional und könnte im späteren Verlauf des Projekts umgesetzt werden.
Um den Nutzer Benachrichtigungen über die Pausen anzuzeigen, wäre ebenfalls eine Firebase Cloud Messaging API für Push-Benachrichtigungen hilfreich. Die Logik hinter dieser Funktion besitzt nicht viel Aufwand und ist mit einer Zähler-Funktion zu vergleichen. Im Vergleich zu anderer Anwendungslogik ist der verbundene Aufwand zu hoch und hat dabei wenig Effekt.

13.6 REST

Für das Anwendungssystem spielt REST eine wichtige Rolle. Mit den folgenden Tabellen wird versucht die Struktur des zu entwickelnden Prototypen durch die Anwendung von HTTP-Methoden zu beschreiben.

Tabelle 16: REST

Ressource	HTTP-Verb	Semantik	Content Type (req)	Content Type (res)
/nutzer	GET	Ausgabe aller Nutzer	text/plain	application/json
/nutzer/{nutzername}	POST	Legt neuen Nutzer an	application/json	text/plain
/nutzer/{nutzername}	PUT	Aktualisiert die Informationen eines Nutzers	application/json	text/plain
/nutzer/{nutzername}	GET	Ausgabe eines bestimmten Nutzers	text/plain	application/json
/karteikarten	GET	Ausgabe aller Karteikarten	text/plain	application/json
/nutzer/karteikarten?nutzer={nutzername}&karteikarte={id}	GET	Ausgabe aller Karteikarten eines bestimmten Nutzers	text/plain	application/json
/nutzer/karteikarten?nutzer={nutzername}&karteikarte={id}	POST	Legt neue Karteikarte für bestimmten Nutzer an	application/json	text/plain
/nutzer/karteikarten?nutzer={nutzername}&karteikarte={id}	PUT	Aktualisiert den Inhalt einer Karteikarte für einen bestimmten Nutzer	application/json	text/plain
/nutzer/karteikarten?nutzer={nutzername}&karteikarte={id}	DELETE	Entfernt eine Karteikarte eines bestimmten Nutzers	application/json	text/plain
/stapel	GET	Ausgabe aller Karteikartenstapel	text/plain	application/json
/nutzer/stapel?nutzer={nutzername}	GET	Ausgabe aller Karteikartenstapel eines bestimmten Nutzers	text/plain	application/json

/nutzer/stapel?nutzer={nutzername}	POST	Legt neue Karteikartenstapel für einen Nutzer an	application/json	text/plain
/nutzer/stapel?nutzer={nutzername}	PUT	Aktualisiert den Inhalt einer Karteikartenstapel eines Nutzer	application/json	text/plain
/nutzer/stapel?nutzer={nutzername}	DELETE	Entfernt einen Karteikartenstapel eines Nutzers	application/json	text/plain
/nutzer/stapel?karteikarte={id}	GET	Ausgabe einer Karteikarte aus einem Karteikartenstapel	text/plain	application/json
/nutzer/stapel?stapel={name}&nutzer={nutzername}	GET	Ausgabe eines öffentlichen Karteikartenstapels von einem Nutzer	text/plain	application/json
/nutzer/merkliste?nutzer={nutzername}	GET	Ausgabe der gespeicherten Karteikartenstapel eines Nutzers	text/plain	application/json
/nutzer/merkliste?nutzer={nutzername}&stapel={name}	GET	Ausgabe eines gespeicherten Karteikarenstapels eines Nutzers	text/plain	application/json
/nutzer/merkliste?nutzer={nutzername}&stapel={name}	POST	Speichern eines Karteikarenstapels in die Merkliste des Nutzers	application/json	text/plain
/nutzer/merkliste?nutzer={nutzername}&stapel={name}	DELETE	Entfernen eines Karteikarenstapels aus der Merkliste des Nutzers	application/json	text/plain

Das Ergebnis zeigt eine REST-Modellierung für den Prototypen des Anwendungssystems und wird folglich im weiteren Verlauf des Projekts angepasst. Während der Bearbeitung wurde darauf geachtet, dass mehrere URIs über eine Ressource ansprechbar und URIs dadurch nicht zu verstreut ist.

13.7 Richardson Maturity Model

Im Richardson Maturity Model wird die REST Architektur des Projekts in 4 Stufen eingeordnet. Durch die REST-Tabelle ist erkennbar, dass das Projekt Stufe 2 einzuordnen ist.
Stufe 0 beschreibt den Gebrauch von HTTP Verben, um eine unidirektionale Kommunikation zu ermöglichen, welches durch die POST-Methode erfüllt wird. Als nächstes beschreibt Stufe 1 den Gebrauch von Ressourcen als Kommunikationsendpunkt, was durch die aufgelisteten URIs der Ressource repräsentiert wird. Da ebenfalls alle 4 HTTP Verben auf verschiedene Ressourcen anwendbar sind, ist somit auch Stufe 2 erfüllt.

Für Stufe 3 ist das Anwenden von Hypertexts nötig, welches zu diesem Zeitpunkt nicht erfüllt wird. Dies schließt jedoch nicht aus, dass Hypertext im weiteren Verlauf der Entwicklung für Verweise außerhalb des Anwendungssystems genutzt werden muss.

13.8 Dienste im Web

Für das Anwendungssystem werden Internetdienste integriert, um die Anwendungslogik implementieren zu können.

Google Maps

Wie zuvor im Konzept beschrieben, wurde sich für eine Google Maps API entschieden, damit Spazierrouten visuell für den Nutzer verfügbar sind.

REST

Im Architekturmodell ist eine Datenbank gegeben, dessen Datenpflege wichtig für die Gebrauchstauglichkeit ist. REST bietet hierfür eine effiziente Möglichkeit, sowohl die Kommunikation zwischen Client und Server zu unterstützen, als auch eine flexible Nutzung der Daten, via HTTP Verben durc Express, zu ermöglichen.
Das Projektteam hat bereits im Studienfach Web Basierte Anwendungen mit REST gearbeitet und sich deshalb bewusst für diesen Dienst entschieden.

Abwägung anderer Dienste

Im Rahmen des Prototyps hat sich das Team gegen die Nutzung weiterer APIs entschieden, da dadurch zwar eine zufriedenstellende Nutzung gewährleistet werden kann, diese aber nicht so viel Priorität hat, wie die erfolgreiche Umsetzung der Anwendungslogik.
Während der Ausarbeitung des Konzepts wurde bereits Firebase Cloud Messaging in Betracht gezogen, um den Nutzer mit Push-Benachrichtigungen über Lernpausen zu informieren. Ebenfalls kann dem Nutzer mitgeteilt werden wie viele Kilometer die Spazierroute noch hat und wie viele Karteikarten für eine Lerneinheit übrig sind. Nach genauerer Betrachtung wurden diese Funktionen mit einer niedrigeren Priorität eingestuft, als das eigentliche Lernsystem der Anwendung. Firebase Cloud Messaging wird weiterhin nicht einbezogen.

13.9 Fehlerbehandlung

Um Fehlerbehandlungen darzustellen werden HTTP-Statuscodes verwendet, die auf ein Request mit einem Statuscode im Header der Response antworten. Im Rahmen der Projektarbeit werden für die aufgelisteten HTTP-Methoden Statuscodes ausgewählt und nach Erfolg oder Misserfolg der Request unterteilt.

- 200 OK: Die Request war erfolgreich.
- 201 Created: Durch die Request wurde eine neue Ressource mit eigener URI angelegt.
- 202 Accepted: Die Request wurde akzeptiert, aber noch nicht verarbeitet.
- 400 Bad Request: Die Request konnte nicht verarbeitet werden.
- 401 Unauthorized: Um die Request zu starten wird eine Nutzer Authentifizierung benötigt.
- 403 Forbidden: Die Request wurde verarbeitet, aber nicht erfüllt.
- 404 Not Found: Für die Request wurde keine richtige URI benutzt.
- 500 Internal Server Error: Unerwarteter Serverfehler.

Die Statuscodes werden die Entwicklung des Anwendungssystems unterstützen und können unter Anderem beim erfolglosen Anlegen eines Karteikartenstapels Aufschluss zu der Fehlerquelle geben.

13.10 Endgeräte

Durch die User Profiles des Vorgehensmodells wurden zuerst die wichtigsten Nutzer des Systems herausgefunden und anschließend mit der Task-Analysis die zugehörigen Funktionen für die Nutzer. Dadurch wurde die Annahme aus dem Konzept, zwei verschiedene Präsentationslogiken anzufertigen, bestätigt.

Für Informatikstudenten und andere Interessenten eines Informatikstudiums steht die Einsicht in Karteikartenstapel im Vordergrund. Von Vorteil ist hierfür eine Smartphone App, die einen standortunabhängigen Zugriff auf das Anwendungssystem ermöglichen sollte. Zur Möglichkeit standen Android und iOs Anwendungen. Es wurde sich früh für eine Android Anwendung entschieden, da beide Teammitglieder Erfahrung beim Programmieren mit Java und dem Entwickeln einer Android Anwendung haben.

Ein Professor der Informatik hingegen möchte Kontrolle über seine Karteikartenstapel haben und Fehler in den preisgegebenen Informationen vermeiden. Um einen besseren Überblick für das Erstellen und Verwalten von Karteikartenstapeln zu haben, bietet sich eine Desktopanwendung an. Wie bereits von der Veranstaltung vorgegeben, wird hierfür eine Umsetzung mit Javascript vorgesehen. Im Konzept wurde bereits beschrieben welche Module für die Implementierung benutzt werden.

14. Reflexion

Das Ziel der Systemdokumentation war es eine Orientierung für den zu entwickelnden Prototypen zu erarbeiten. Da die Systemarchitektur im Konzept fehlinterpretiert wurde und die Anwendungslogik ebenfalls unklar war, sollte durch eine Iteration klare Ziele gesetzt werden.
Nach der Fertigstellung der Artefakte wurde vom Projektteam festgestellt, dass diese Ziele nicht vollkommen, aber ausreichend für den Prototypen erreicht wurden. Eindeutige Anweisungen für den weiteren Verlauf wurden nicht gefunden, aber Schritte, die als Richtwerte für den weiteren Verlauf dienen können. Im Rahmen der Anwendungslogik wurde sich mit Möglichkeiten für die Umsetzung der Formeln auseinandergesetzt. Durch die festgelegten Datenstrukturen konnte Klarheit von den hier angesprochenen Attributen geschaffen werden.

Obwohl ein detailreicher Zeitplan zu Beginn aufgestellt wurde, gab es während der Bearbeitung der Artefakte viele Hindernisse, die den Arbeitsfluss störten. Das Team hat sich vorgenommen neben der Dokumentation die Programmierung fortzuführen, um somit Artefakte detailreicher darstellen zu können und mehr Bezüge auf die Umsetzung zu nehmen. Dies konnte nicht eingehalten werden.

Für den weiteren Projektverlauf sollte geklärt werden, welche Komponenten der Systemarchitektur umgesetzt werden. Hierbei ist besonders das Ausmaß der Anwendungslogik wichtig, um den Erwartungen der Abnahme gerecht zu werden.

15. Quellenverzeichnis

1 Mayhew, Deborah. 1999. The Usability Engineering Lifecycle: A Practitioner's Handbook for User Interface Design
2 https://docs.microsoft.com/de-de/windows/uwp/design/basics/ (aufgerufen am 10.11.2018)
3 https://material.io/design/introduction/#principles (aufgerufen am 10.11.2018)
4 Lennken, Revista. 2013. Design Methods

16. Abbildungsverzeichnis

Kapitel	Beschreibung	Seite
1.2	Abb. 1: Usability Engineering Lifecycle nach Deborah Mayhew	05
2	Abb. 2: Top-Down Modell Findung einer Subdomäne	07
5.3	Abb. 3: Current User Task Organization Model Lernprozess	19
6.2	Abb. 4: Usability Goal 1	23
6.2	Abb. 5: Usability Goal 2	24
6.2	Abb. 6: Usability Goal 3	24
9.2	Abb. 7: Task Scenario Spazier-Route	35
9.2	Abb. 8: Task Scenario Abspeichern	36
9.3	Abb. 9: Reengineered Task Organization Model	37
10.4	Abb. 10: Conceptual Model Desktopanwendung	40
10.4	Abb. 11: CM1 Androidanwendung	41
10.4	Abb. 12: CM2 Androidanwendung	41
12.1	Abb. 13: Startscreen calmRunter	46
12.1	Abb. 14: Seitenmenü calmRunter	46
12.1	Abb. 15: Lerneinheit calmRunter	47
12.1	Abb. 16: Starten calmRunter	47
12.1	Abb. 17: Community calmRunter	48
12.1	Abb. 18: Karteikartenstapel calmRunter	48
12.1	Abb. 19: Karteikarte calmRunter	49
12.1	Abb. 20: Karteikarten calmRunter	49
12.2	Abb. 21: Desktopanwendung calmRunter	50
13.1	Abb. 22: Deskriptives Kommunikationsmodell	51
13.1	Abb. 23: Präskriptives Kommunikationsmodell	52
13.2	Abb. 24: Iteration der Systemarchitektur	53
13.4	Abb. 25: Datenstruktur Karteikarte	57
13.4	Abb. 26: Karteikartenstapel Datenstruktur	58
13.4	Abb. 27: Nutzer Datenstruktur	59

17. Tabellenverzeichnis

Kapitel	Beschreibung	Seite
4.1	Tabelle 1: User Profile: Informatikstudent	10
4.2	Tabelle 2: User Profile: Schüler mittlerer Reife	11
4.3	Tabelle 3: User Profile: Professor der Informatik	12
5.1	Tabelle 4: Brief Use Case: Lernende	14
5.1	Tabelle 5: Brief Use Case: Lehrende	15
7	Tabelle 6: Plattform: Desktopanwendung	26
7	Tabelle 7: Plattform: Android Anwendung	27
9.1	Tabelle 8: Use Case 1: Karteikartenstapel auswählen	30
9.1	Tabelle 9: Use Case 2: Lerneinheit abbrechen	31
9.1	Tabelle 10: Use Case: Karteikartenstapel der Merkliste hinzufügen	31
9.1	Tabelle 11: Use Case: Merkliste ansehen	32
9.1	Tabelle 12: Use Case:Statistiken ansehen	33
91.	Tabelle 13: Use Case:Statistiken ansehen	33
9.1	Tabelle 14: Use Case:Statistiken ansehen	34
11.1	Tabelle 15: Mobile Menu Content	43
13.6	Tabelle 16: REST	63